Table of Contents

Campus Recruitment Command Center

An Integrated System for Student Placements & Corporate Relations

PAWAN KUMAR RAJAK

Problem Statement

Project Title: "Campus Recruitment Command Center - An Integrated System for Student Placements & Corporate Relations"

Industry: Higher Education

Project Type: B2S (Business-to-Student) & B2B (Business-to-Business) Salesforce Platform

Target Users: Training & Placement (T&P) Officers, Students, and the Head of Placements

Problem Statement

A university's Training & Placement (T&P) cell manages hundreds of corporate relationships and thousands of student applications using disconnected tools like spreadsheets, Google Forms, and email. This manual process results in communication gaps, missed application deadlines for students, and an inability for T&P officers to efficiently match the right

students with the right opportunities. Furthermore, the constant influx of student queries via email overwhelms the T&P staff, leading to slow response times for critical questions.

To address this, the university wants to implement a Salesforce platform to:

- Centralize company and job posting information.
- Automate the student application and notification process.
- Streamline and automate the resolution of common student queries.
- Provide real-time dashboards for the Head of Placements to track success metrics.

Use Cases

1. Corporate & Recruiter Management

- Maintain a unified database of all visiting companies and their recruiter contact information.
- Track the status of engagement with new companies (e.g., Approached, MoU Signed, Visit Scheduled).
- Log all interactions, such as calls and meetings, with company representatives.

2. Job Posting & Application Management

- Create and manage detailed Job Postings with specific eligibility criteria (e.g., Branch, GPA, Graduation Year).
- Automatically notify eligible students via email when a new relevant job is posted.
- Allow students to apply for jobs through a simple, guided interface (a Screen Flow).
- Track the status of each student's application (e.g., Applied, Shortlisted, Offer Extended).

3. Automated Student Query Resolution

- Automatically create a "Query" record in Salesforce when a student sends an email to a designated placement helpdesk address (e.g., placements@university.edu).
- Send an automated initial email response acknowledging receipt of the query.
- Use a Flow to scan the query's subject for keywords (e.g., "Resume," "Eligibility," "Deadline") and send back a templated email with helpful links or standard answers, closing common queries without manual intervention.

4. Reporting & Analytics

- A master dashboard for the Head of Placements showing key metrics like:

    - Number of Companies Visiting
    - Total Job Openings vs. Applications
    - Branch-wise Placement Percentage
- Reports that track the placement funnel and T&P officer activity.

Phase 1: Problem Understanding & Industry Analysis

Date: September 10, 2025

1. Requirement Gathering

The primary goal of this project is to transform the university's manual and fragmented placement process into a streamlined, automated, and centralized system using the Salesforce platform.

Functional Requirements (What the system must do):

- **Company & Recruiter Management:** A central database to store and manage information about visiting companies (Accounts) and their recruiters (Contacts).
- **Job Posting Management:** T&P officers must be able to create, update, and manage job postings with specific eligibility criteria (e.g., Branch, GPA, required skills).
- **Automated Student Notifications:** The system must automatically email eligible students as soon as a new, relevant job is posted.
- **Student Application Portal:** Students need a simple, guided way to view and apply for open positions.
- **Application Tracking:** The system must track the status of each student's application (Applied, Shortlisted, Interviewing, Offer Extended, Rejected).
- **Automated Query Handling:** The system must automatically create a case/record from incoming student emails and provide an automated, templated response for common queries.
- **Reporting & Analytics:** The Head of Placements requires real-time dashboards to view key metrics like placement rates, company engagement, and application numbers.

Technical Requirements:

- The solution must be built on the Salesforce Lightning Platform.
- The system must enforce data security through a role-based model (T&P Officer vs. Student).

- The solution should be scalable to handle data for thousands of students and hundreds of companies.

2. Stakeholder Analysis

The success of the project depends on meeting the needs of its key users.

Stakeholder Role	Needs & Goals	Current Pain Points
T&P Officer	- Quickly add new companies and jobs. Reduce manual data entry and email tasks.Track all student applications in one place.	- Data is scattered across multiple spreadsheets. Manually filtering and emailing students is time-consuming and error-prone.Overwhelmed by repetitive student questions.
Student	- A single, reliable source for all job openings. Timely alerts for jobs they are eligible for. A clear and simple application process.	- Fear of missing deadlines due to communication gaps. Confusion from information on different platforms (email, WhatsApp, website). Slow response times for important queries.
Head of Placements	- High-level, real-time view of placement statistics. Data-driven insights to improve corporate relations. Professional reports for university management.	- Lack of accurate, real-time data for strategic decision-making. Reporting is a manual and time-consuming process at the end of the placement season.

3. Business Process Mapping

As-Is Process (Current State):

1. A company recruiter emails job details to a T&P Officer.
2. The T&P Officer manually checks a master student spreadsheet to find eligible candidates.

3. The Officer composes and sends a bulk email to the filtered list.
4. Students apply by replying to the email with their resumes.
5. The T&P Officer tracks these applications in another spreadsheet.
6. This process is slow, inefficient, and creates multiple versions of the truth.

To-Be Process (Future State with Salesforce):

1. A T&P Officer creates a Job Posting record in Salesforce with defined eligibility criteria.
2. **SYSTEM ACTION:** A Record-Triggered Flow instantly sends a templated email to all Contact records (Students) who meet the criteria.
3. The student clicks a link in the email, launching a Screen Flow to apply.
4. **SYSTEM ACTION:** Upon flow completion, an Application record is automatically created, linking the Student and the Job Posting.
5. All data is instantly available on a real-time dashboard for the Head of Placements.

4. Industry-specific Use Case Analysis (Higher Education)

In the Higher Education sector, the Training & Placement cell is a critical function that directly impacts the university's reputation and ranking. This project addresses a core industry challenge: bridging the gap between academia and the corporate world effectively.

- **Enhancing the Student Experience:** A smooth, transparent, and professional placement process is a key differentiator for universities. This system directly enhances the student experience at the most critical final stage of their journey.
- **Strengthening Corporate Partnerships:** Providing corporate partners with a streamlined recruiting process makes the university a more attractive place to hire from, leading to better and more numerous job opportunities for students.
- **Data-Driven Strategy:** This system moves the T&P cell from a purely administrative function to a strategic one, allowing the Head of Placements to use data to identify trends and improve their outreach strategies.

5. AppExchange Exploration

A review of the Salesforce AppExchange was conducted to identify existing solutions.

- **Salesforce Education Cloud:** This is Salesforce's flagship product for the education industry. It is an extensive, enterprise-level solution that manages the entire student lifecycle, from admissions to alumni relations. While powerful, it is too large and complex for the specific, immediate needs of the T&P cell and the 5-day project scope.

- **Third-Party Apps:** Various apps exist for event management, email marketing, and form building.

Conclusion: A solution that is perfectly tailored to the university's unique placement process, can be developed rapidly, and avoids the cost and complexity of a large, pre-built package. This project will deliver maximum value by focusing directly on solving the T&P cell's most pressing problems.

Phase 2: Org Setup & Configuration Summary

Date: September 11, 2025

1. Core Company Configuration

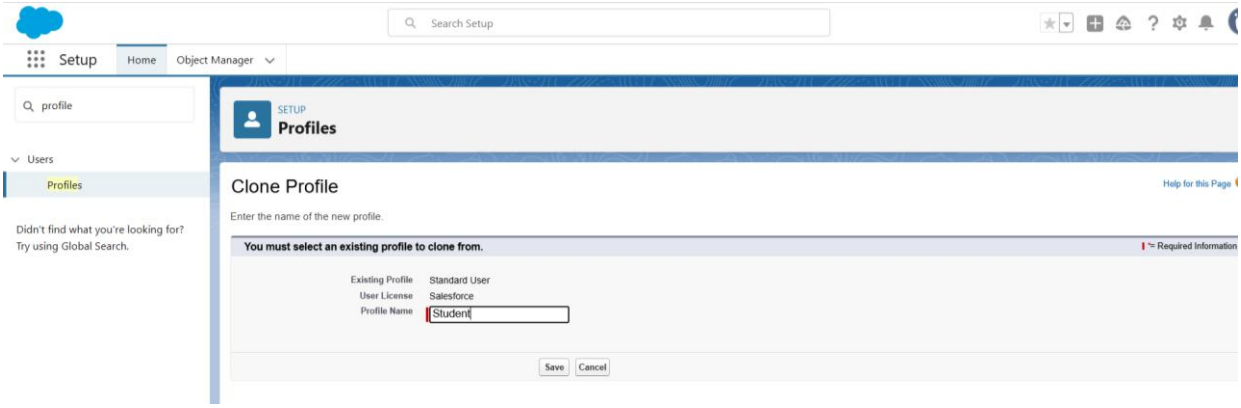
The foundational settings for the Salesforce org were established to mirror the operational environment of the university's T&P Cell.

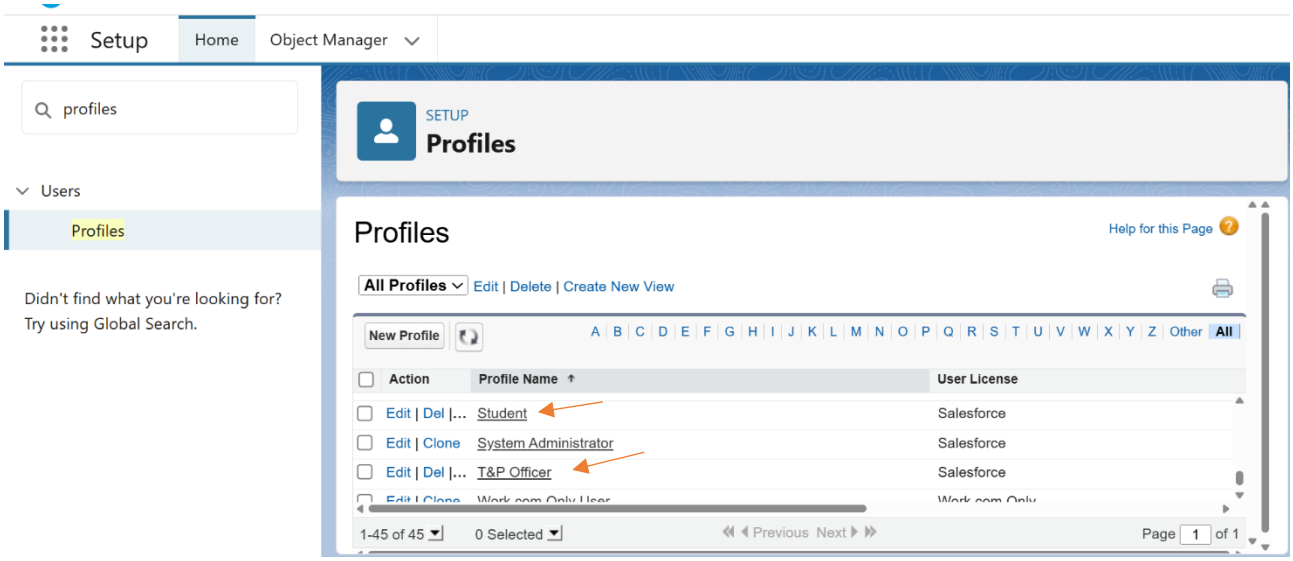
- **Company Information:** The org was configured with the university's name, default time zone (Asia/Kolkata), and locale to ensure all time-stamped data is accurate.
- **Business Hours & Fiscal Year:** Standard business hours (Mon-SAT, 12 AM - 12 AM) were set to enable future tracking of case/query response times. A standard fiscal year was confirmed for reporting consistency.
- **Developer Org:** A new Salesforce Developer Edition org was created to serve as the dedicated environment for this project.

2. Security Model Implementation

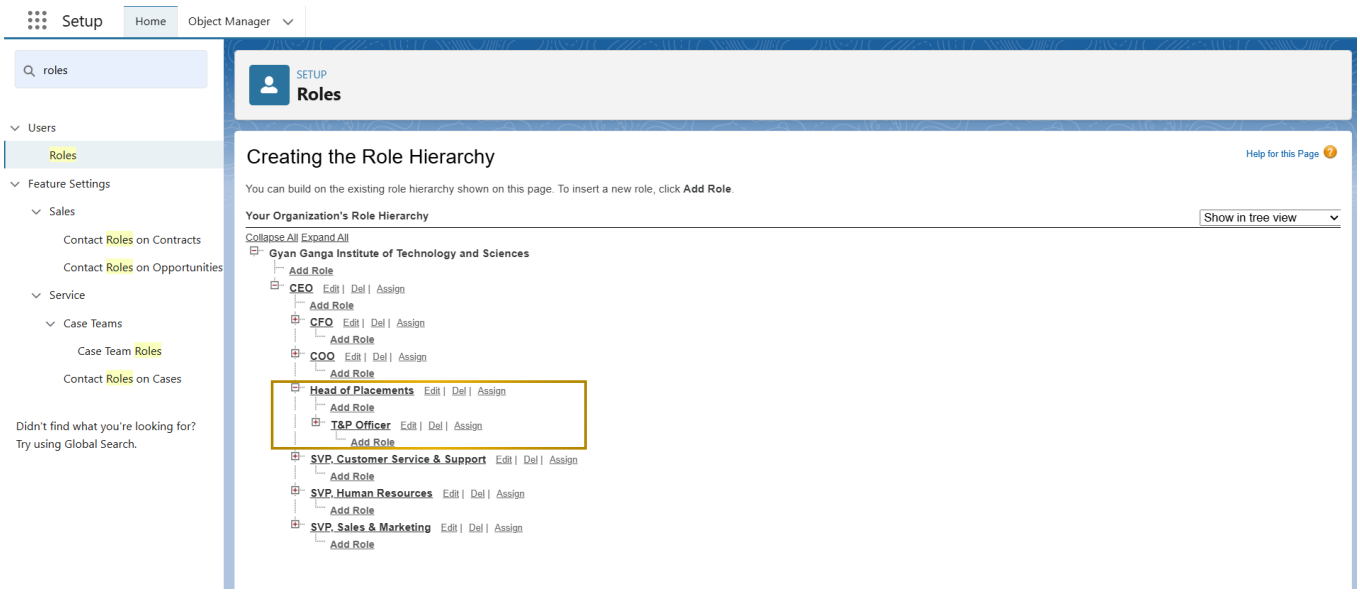
A multi-layered security model was configured to ensure data is accessible only to the appropriate users.

- **Profiles:** Two custom profiles were created by cloning the "Standard User" profile:
    - T&P Officer: For staff who will manage the system.
    - Student: For end-users who will interact with the system.

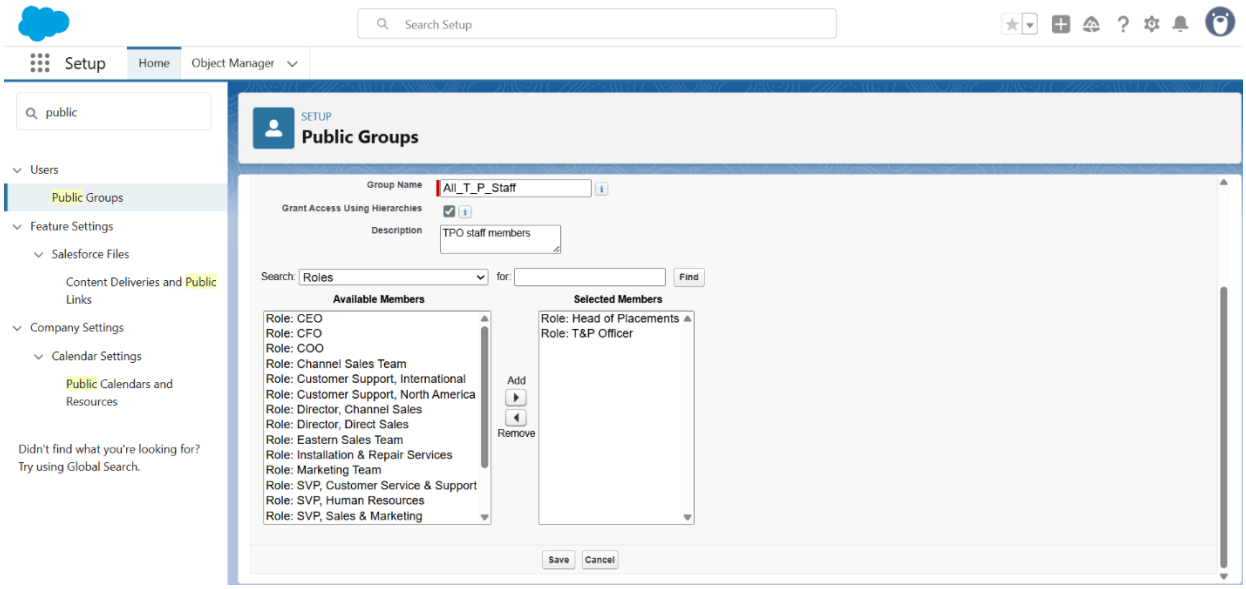




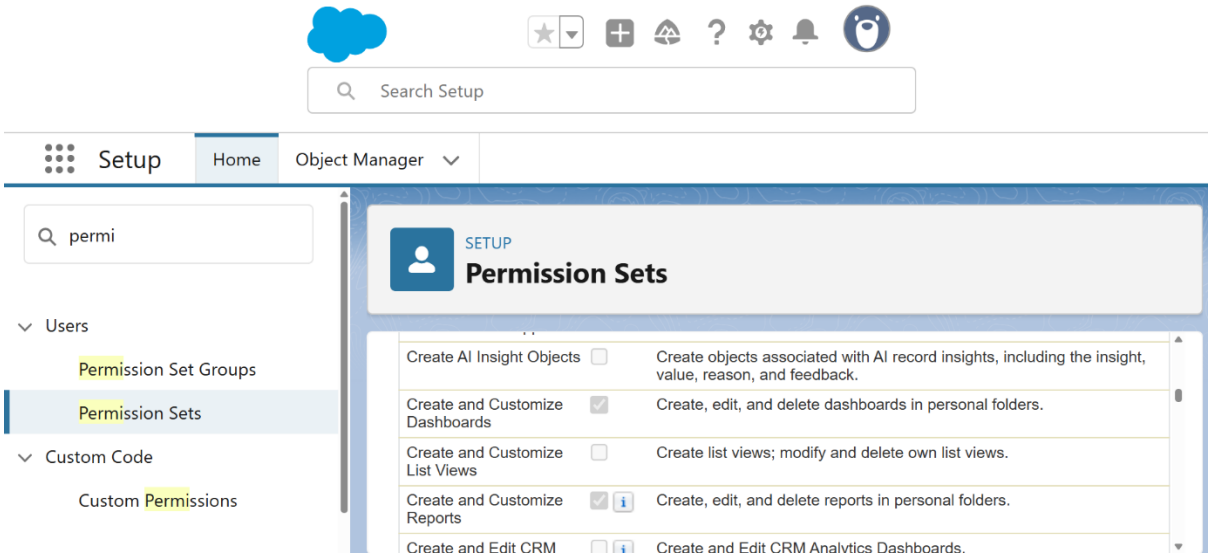
- **Role Hierarchy:** A top-down data visibility structure was created: **CEO -> Head of Placements -> T&P Officer**. This ensures management can view the records of their subordinates.



- **Organization-Wide Defaults (OWD):** A **Private** data model was established as the baseline. This is a security best practice, ensuring users can only see their own records by default unless explicitly granted access.



- **Permission Set:** A Permission Set named T&P Analytics Manager was created to grant specific, additive permissions (e.g., "Create and Customize Dashboards") to select users without changing their entire profile. This demonstrates a flexible approach to security.



3. Deferred Configurations & Justification

Certain configurations mentioned in the project checklist were intentionally deferred to a later phase for strategic reasons.

- **Sharing Rules:** Sharing Rules are used to open up access to records that are locked down by a Private OWD. These rules are record-specific. **Reason for Deferral:** The custom objects (Job Posting, Application, T&P Query) have not been created yet. Sharing Rules for these objects will be configured in Phase 4 after the data model is built.
- **Login IP Ranges:** This security feature restricts user logins to specific IP addresses. **Reason for Deferral:** During the development phase, it is impractical to restrict IP ranges as development may occur from various locations. This is a production-level security measure that would be implemented post-development, before final deployment.
- **Sandbox Usage & Deployment:** These concepts relate to the development lifecycle in a team environment. **Reason for Deferral:** As a rapid, 5-day solo project, development is being done directly in a Developer Org for maximum efficiency. In a real-world team project, a sandbox-driven development and deployment strategy would be utilized.

Phase 3: Data Modeling & Implementation Report

Date: September 12, 2025

1. Overview

This document outlines the actions taken during the Data Modeling phase of the project. The objective was to create a robust, scalable, and secure data structure to support all functionalities of the Training & Placement (T&P) application. This involved configuring standard Salesforce objects, creating new custom objects and fields, establishing relationships, and enhancing the user interface.

2. Standard Object Configuration

Standard Salesforce objects were leveraged and configured to fit the project's specific terminology and data segregation needs.

- **Account Object:**

- **Action:** The Account object was renamed to Company.
- **Justification:** This provides a more intuitive user experience for T&P Officers, who primarily manage relationships with companies, not "accounts."

- **Contact Object (with Record Types):**
  - **Action:** Two record types were created on the Contact object: Student and Recruiter. Each record type was assigned a unique page layout (Student Layout and Recruiter Layout).
  - **Justification:** The Contact object stores two distinct types of people. Record Types are the Salesforce best practice for segregating them, allowing for different data fields and user experiences on the same object.
- **Lead Object:**
  - **Action:** The standard Lead object was not renamed or configured for this project's initial scope.
  - **Justification:** The primary focus is on managing relationships with established corporate partners (Companies) and student applications. While future enhancements could use Leads for tracking potential new corporate partners, it is not part of the core requirement for this 5-day build.

3. Custom Object & Field Creation

Three core custom objects were created to form the backbone of the application.

Object 1: Job Posting (Job_Posting__c)

Object 1: Job Posting (Job_Posting__c)

Field Label	API Name	Data Type	Details
Job Posting Name	Name	Text	The primary identifier/title of the job.
Company	Company__c	Master-Detail	A required link to the parent Company (Account) record.
Description	Description__c	Text Area (Long)	For the detailed job description.
Location	Location__c	Text	The city/location of the job.
Package (CTC)	Package_CTC__c	Currency	The offered Cost-to-Company.

Field Label	API Name	Data Type	Details
Eligibility Branch	Eligibility_Branch__c	Picklist (Multiselect)	Values: Computer Science, Mechanical, Electronics, Civil.
Minimum GPA	Minimum_GPA__c	Number (2,1)	Required GPA for eligibility.
Status	Status__c	Picklist	Values: Open, Interviewing, Closed.

Object 2: Application (Application__c) – Junction Object

Field Label	API Name	Data Type	Details
Application ID	Name	Auto-Number	Format: APP-{0000}.
Job Posting	Job_Posting__c	Master-Detail	A required link to the parent Job_Posting__c record.
Student	Student__c	Master-Detail	A required link to the parent Contact (Student) record.
Status	Status__c	Picklist	Values: Applied, Shortlisted, Interview Scheduled, Offer Extended, Rejected.

Object 3: T&P Query (T_P_Query__c)

Field Label	API Name	Data Type	Details
Query ID	Name	Auto-Number	Format: Q-{00000}.
Student	Student__c	Lookup	An optional link to the Contact (Student) record.
Description	Description__c	Text Area (Long)	The full text of the student's question.
Status	Status__c	Picklist	Values: New, Automated Reply Sent, Manual Follow-up Needed, Closed.

Field Label	API Name	Data Type	Details
Category	Category__c	Picklist	Values: Eligibility, Resume, Deadline, Technical Issue, Other.

---

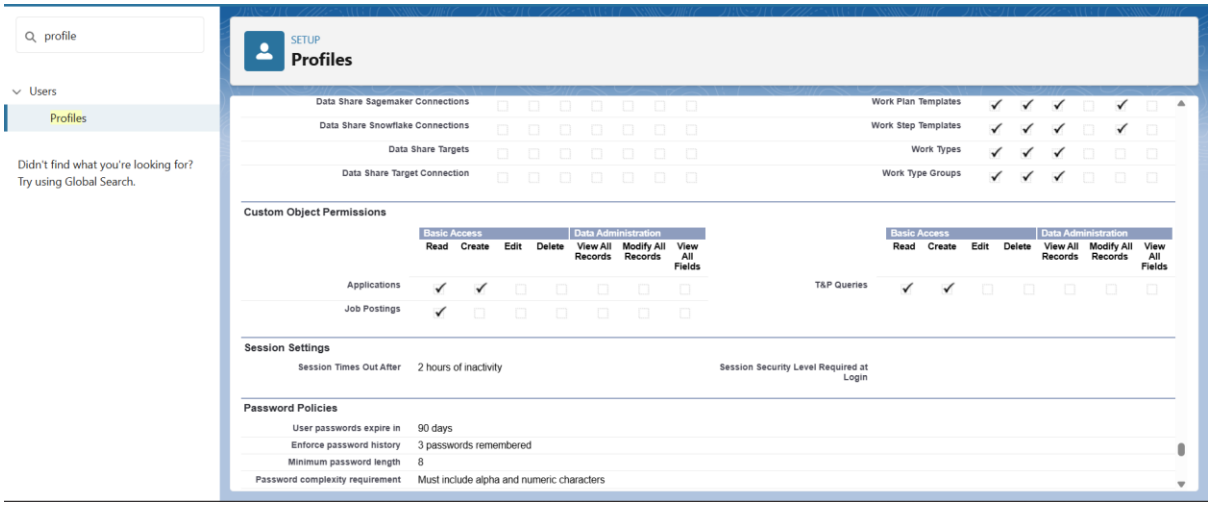
4. Relationship Decisions (Master-Detail vs. Lookup)

The choice of relationship type is critical for data integrity.

- **Master-Detail Relationship (MDR):** Used on the Application object for its links to Job Posting and Contact.
    - **Justification:** An application is fundamentally dependent on both a job and a student. It cannot exist on its own. The MDR enforces this by ensuring an application record is automatically deleted if either its associated job or student is deleted (cascading delete). This also allows for rollup summary fields in the future.
- **Lookup Relationship:** Used on the T&P Query object to link to the Contact.
    - **Justification:** A query is related to a student, but it is also a historical record for the T&P cell. If a student's record is deleted after graduation, the T&P cell may still want to retain the query for analytics. A Lookup provides a looser connection that prevents the query from being auto-deleted, thus preserving historical data.

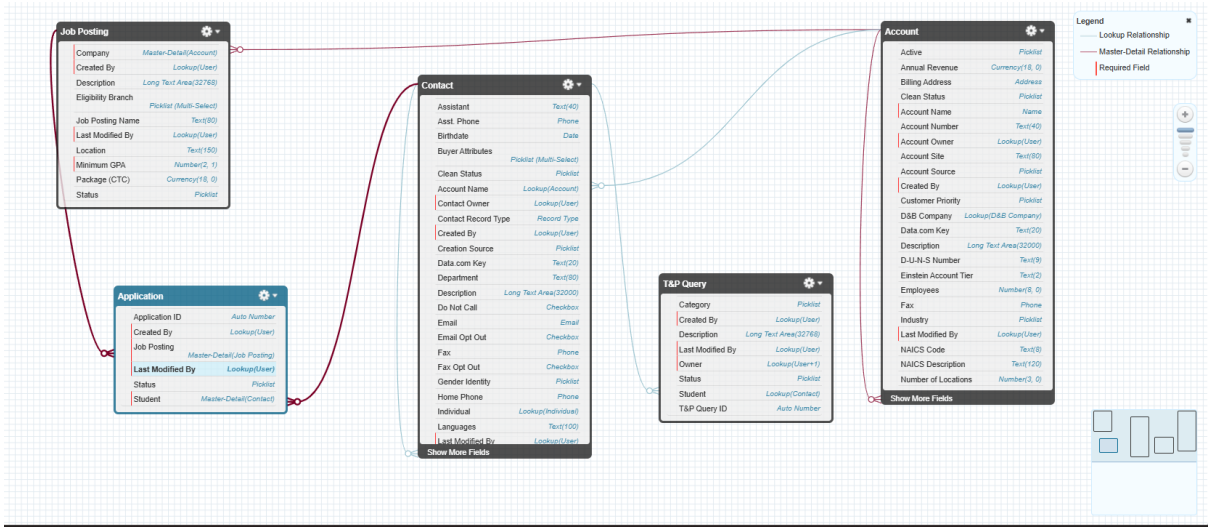
5. User Interface & Field-Level Security

- **Page Layouts & Compact Layouts:** Custom page layouts were created for the Student and Recruiter record types. A custom Compact Layout, Job Posting Highlights, was created to show key information at the top of the job posting page.
- **Field-Level Security (FLS):** FLS was configured to ensure users only see relevant data.
    - **User-Driven Modification:** Based on user requirements for transparency, the Package (CTC)__c field on the Job Posting object has been made **Read-Only** for the Student profile. This allows students to make informed decisions while preventing them from editing the data.
    - **Internal Fields:** The Category__c field on the T&P Query object was **hidden** from the Student profile as it is an internal classification field for T&P officers only.



6. Schema Builder Visualization

The Schema Builder was used to view the complete data model. This provides a clear, visual blueprint of all objects and their intricate relationships, confirming the logical integrity of the application's architecture.



7. Excluded Concepts

- **Hierarchical & External Objects:** These relationship types were not applicable. Hierarchical relationships are specific to the User object, and External Objects are for integrating data from outside systems, which is beyond the scope of this self-contained project.

Phase 4: Process Automation Implementation Final Report

Date: September 17, 2025

1. Overview

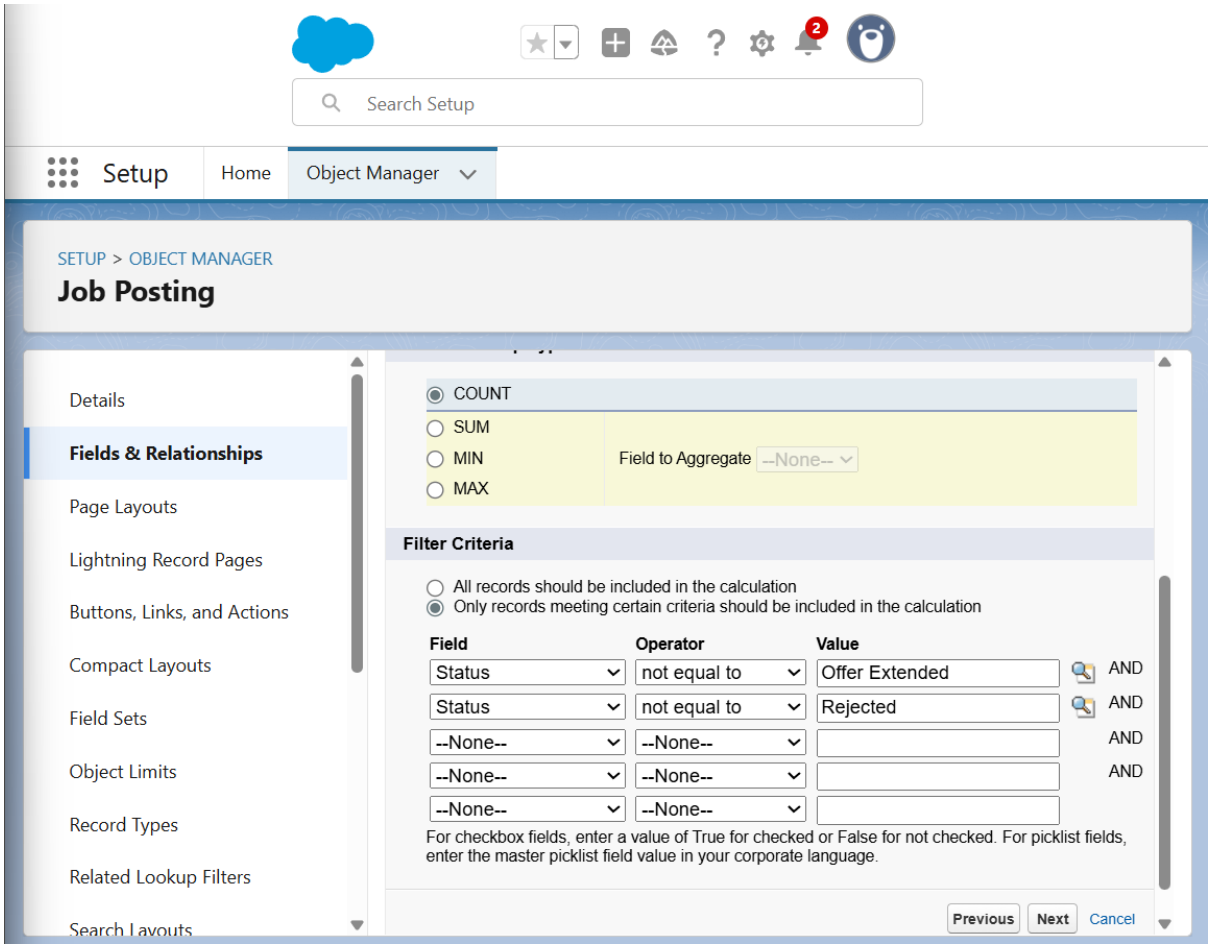
This document provides a comprehensive summary of the process automation implemented during Phase 4. Building upon the secure data model from the previous phases, the focus was to embed the T&P Cell's business logic directly into the application. This was achieved by creating a suite of automations that enforce data quality, guide user actions, eliminate manual work, and ensure timely, professional communication. All automations were built using Salesforce Flow, adhering to modern best practices for scalability and maintenance.

2. Implemented Automations & Processes

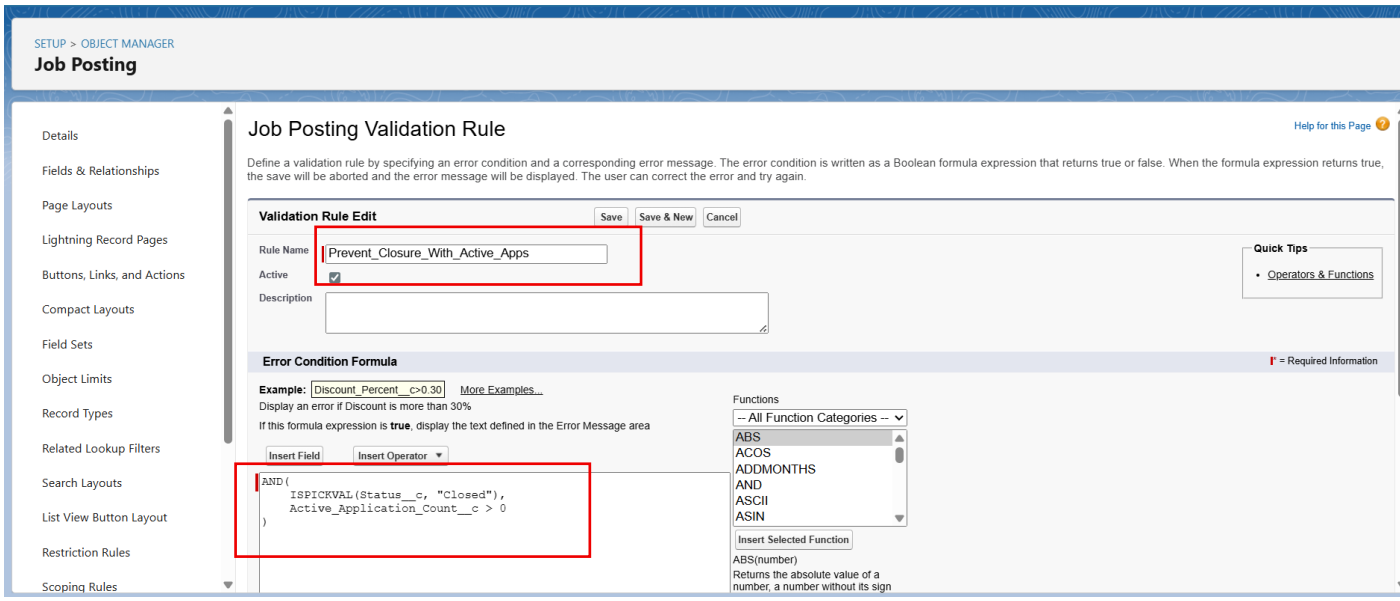
A. Validation Rule: Preventing Data Inconsistency

A validation rule was implemented to protect data integrity and enforce a critical business process.

- **Business Need:** The T&P Cell cannot officially close a Job Posting if there are still students with active applications for that role. This prevents administrative errors and ensures no student is left in limbo.
- **Implementation Steps:**
    1. A **Roll-Up Summary Field** named Active Application Count was created on the Job Posting object. This field provides a real-time count of all related Application records that are not in a final state (Offer Extended or Rejected).



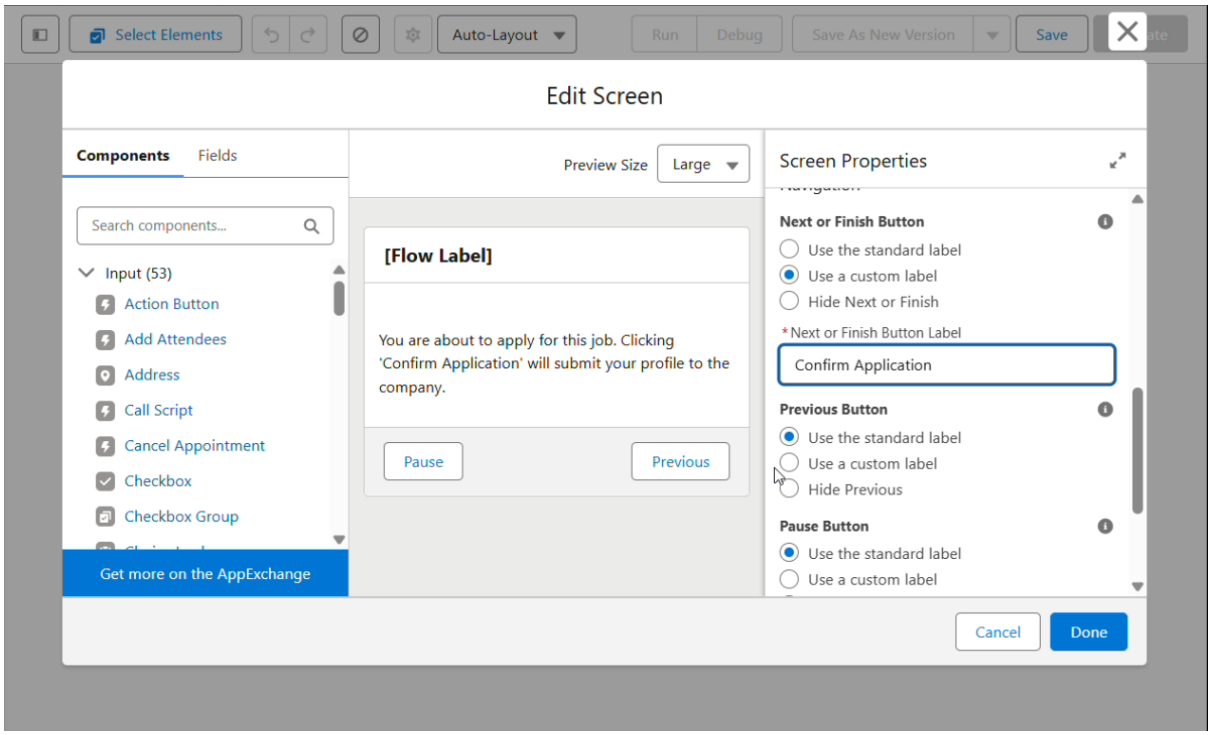
2. A **Validation Rule** was then created on the Job Posting object. This rule checks two conditions upon saving a record: if the Status is being changed to "Closed" AND if the Active Application Count is greater than 0. If both are true, the save action is blocked, and a clear error message is displayed to the user.

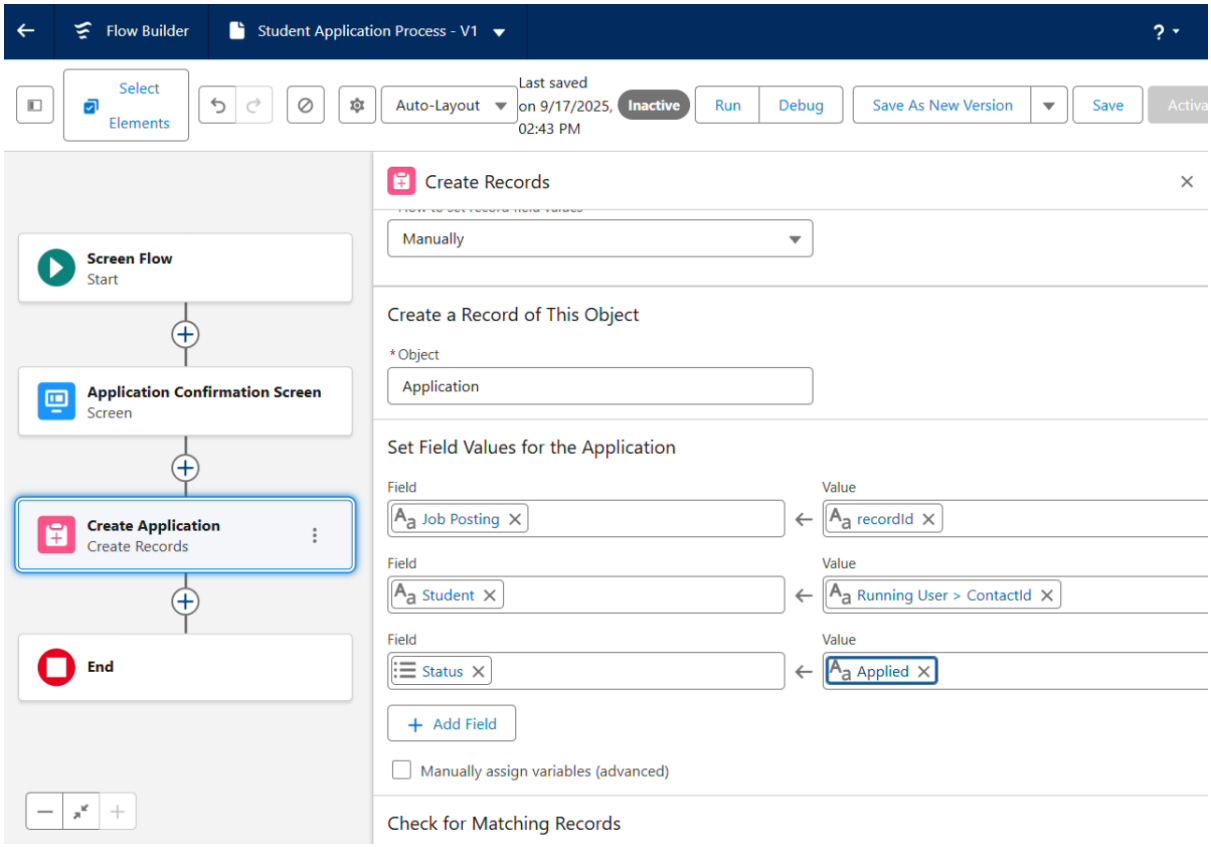


B. Screen Flow: A Guided Application Experience for Students

To enhance the student user experience and streamline the application process, a guided screen flow was developed.

- **Business Need:** Students require a simple, intuitive, and error-proof method to apply for jobs. A manual process of creating records would be inefficient and confusing.
- **Implementation Steps:**
    1. A **Screen Flow** titled "Student Application Process" was built. It features a simple confirmation screen to ensure the student's intent.
    2. The flow is intelligent, automatically capturing the recordId of the Job Posting it was launched from and the ContactId of the logged-in student user ($User.ContactId).
    3. Upon confirmation, a Create Records element automatically generates a new Application record, correctly linking the student and job, and setting the initial Status to "Applied".
    4. This flow was then made accessible by creating a **Quick Action** button labeled "Apply Now" and adding it to the Job Posting page layout, ensuring high visibility.

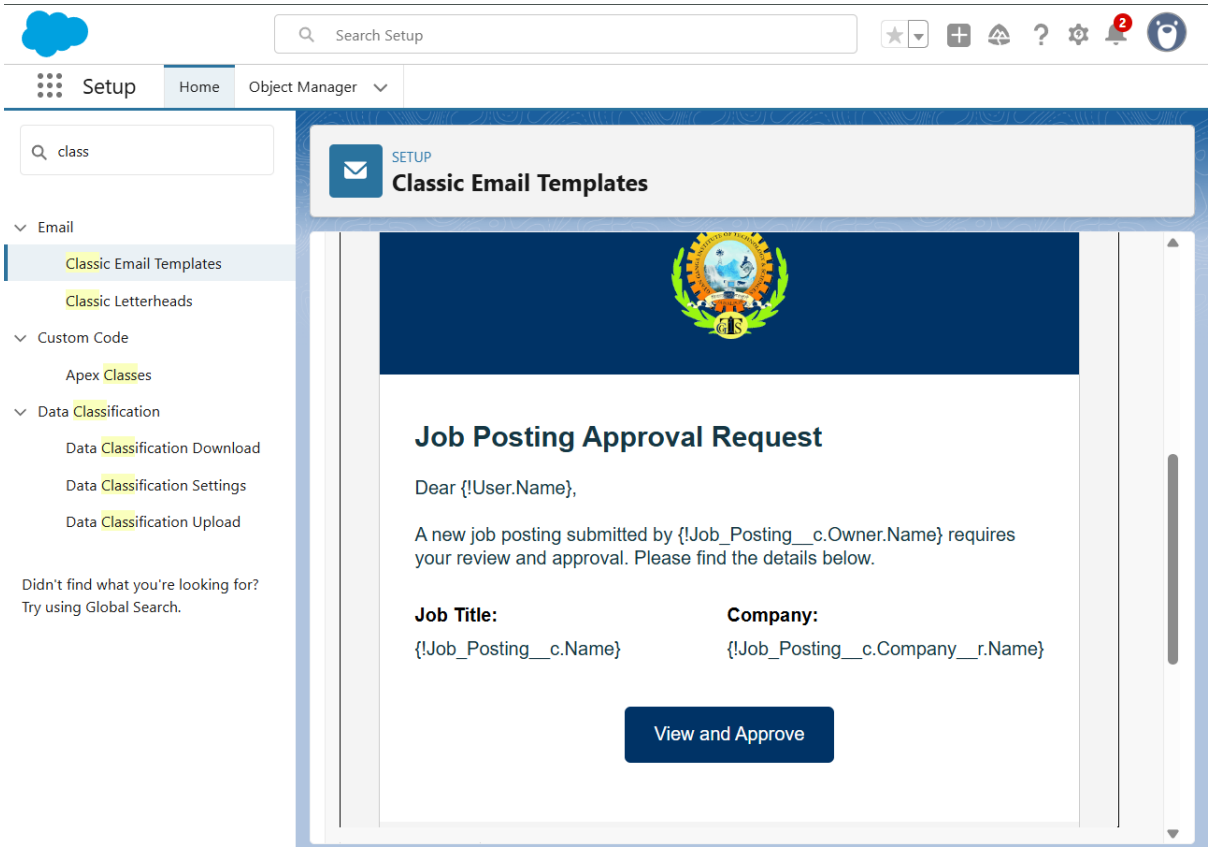




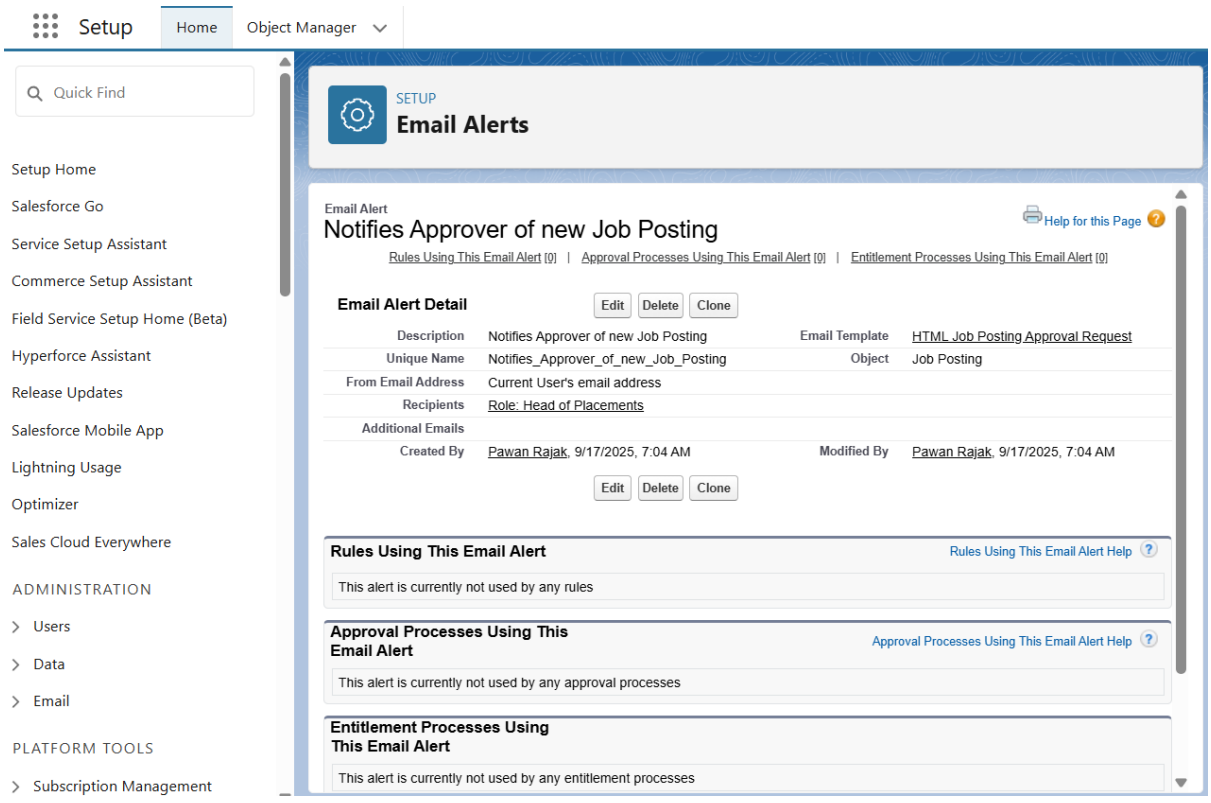
C. Approval Process & HTML Email Alert: Formalizing Job Postings

A formal approval process, supported by a professional HTML email alert, was created to ensure management oversight.

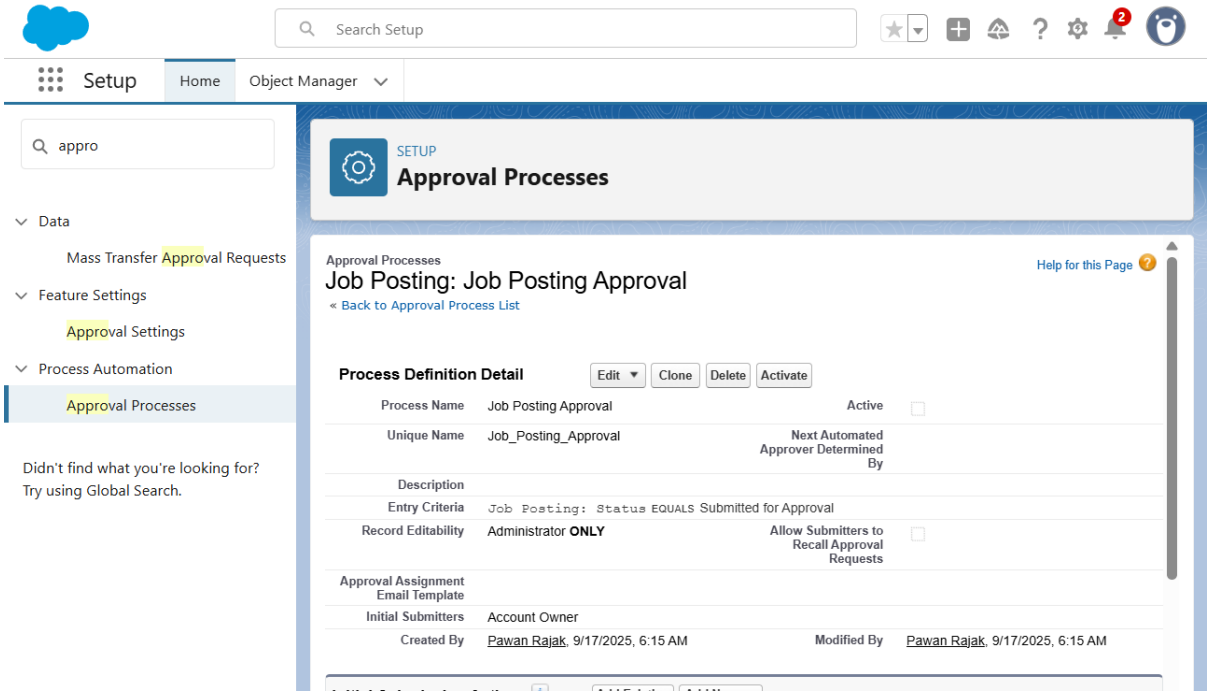
- **Business Need:** All new job postings must be reviewed and approved by the Head of Placements before they are made public to students, ensuring quality and consistency. The notification for this approval must be professional and actionable.
- **Implementation Steps:**
    1. An **HTML Email Template** was created to provide a branded, easy-to-read notification. The template includes the university logo, merges in key details like the Job Title and Company, and features a prominent "View and Approve" button that links directly to the record in Salesforce.



2. An **Email Alert** action was configured to use this HTML template and send it to any user with the Head of Placements role.

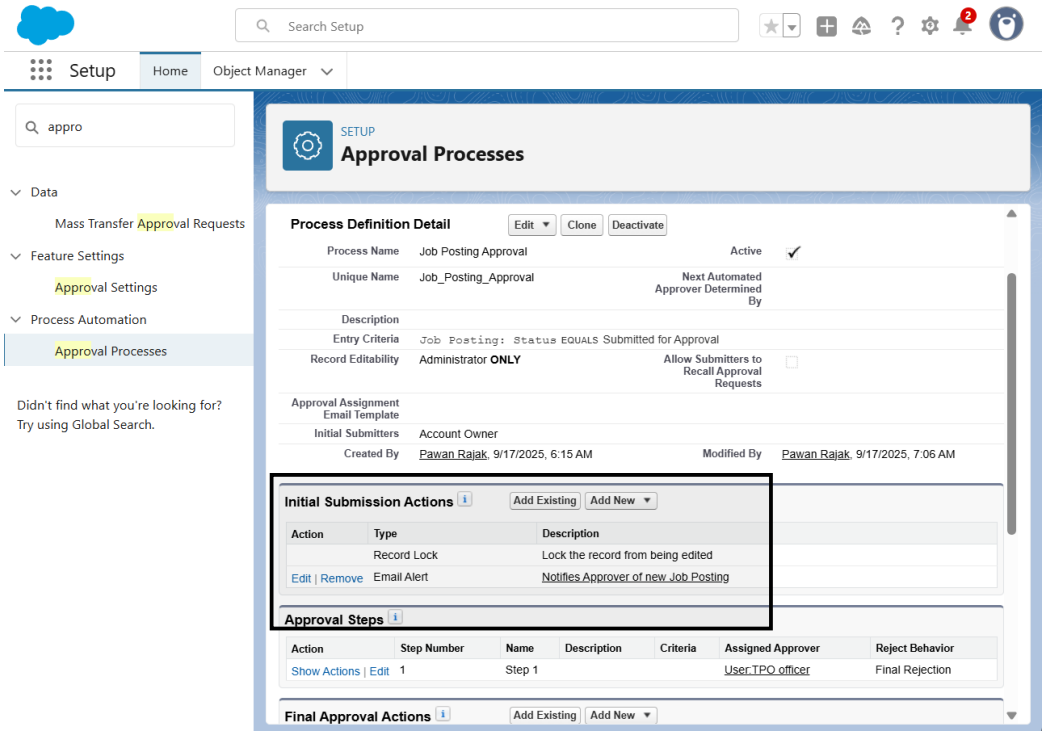


3. An **Approval Process** was configured on the Job Posting object. It triggers when a record's Status is set to "Submitted for Approval".

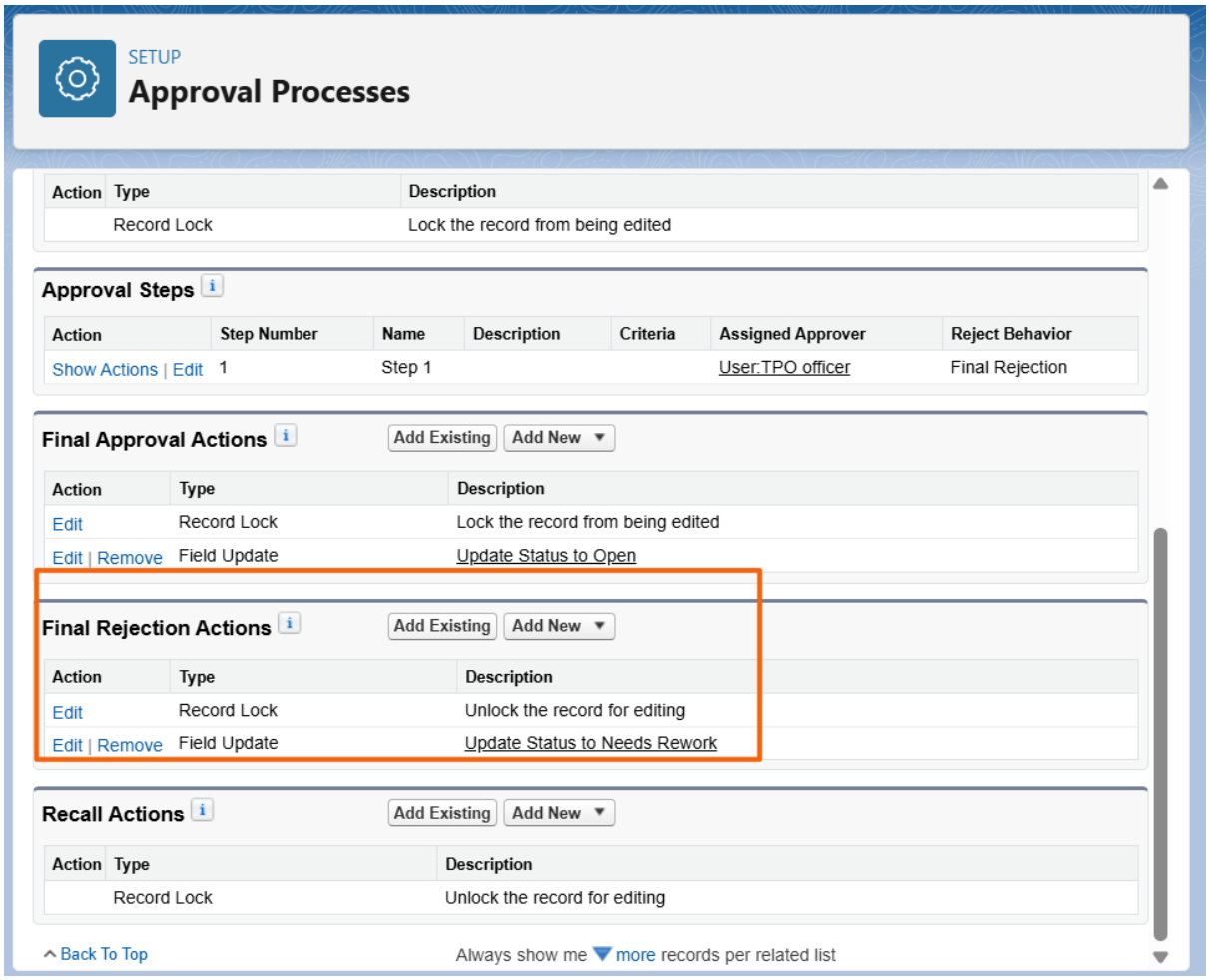


4. The approval request is automatically assigned to the submitter's manager via the Role Hierarchy.

5. The Email Alert was added to the "Initial Submission Actions," ensuring the approver is notified instantly.



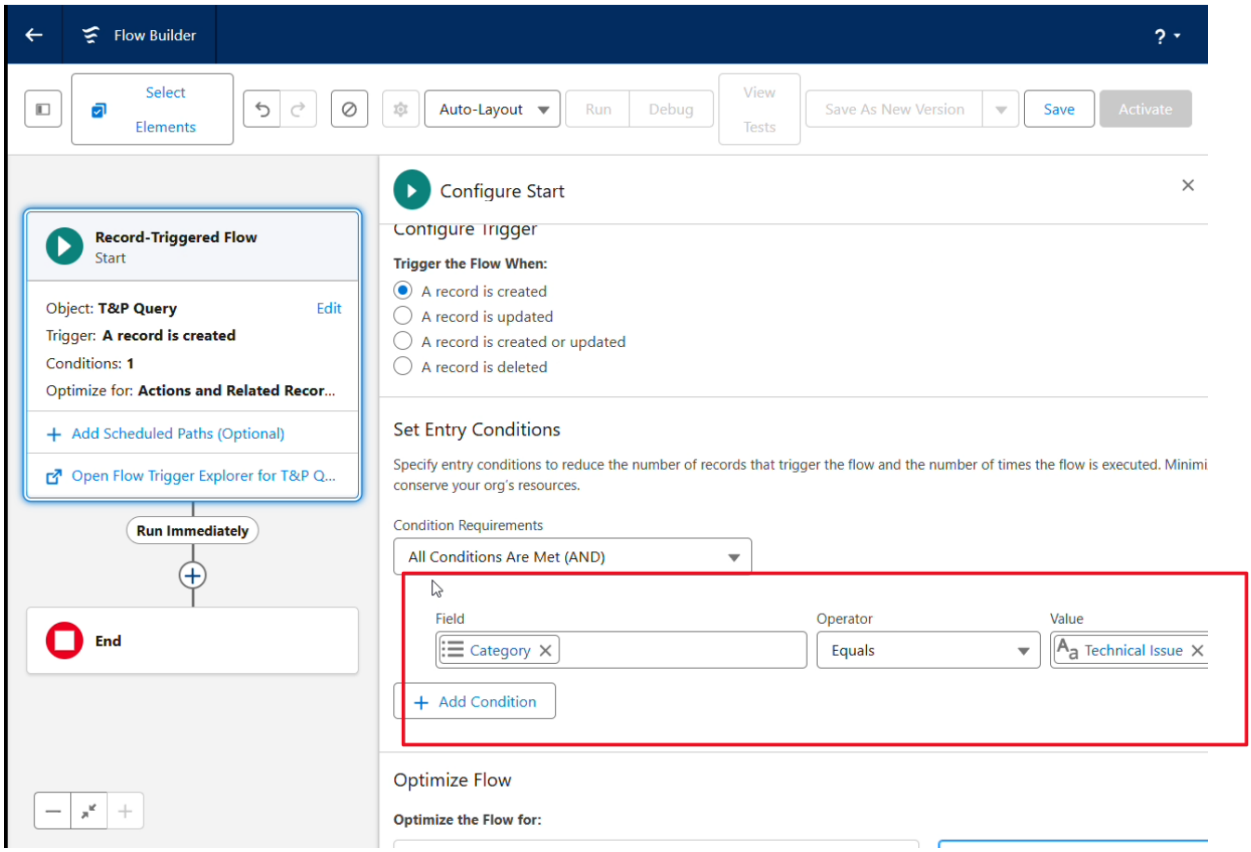
6. Field Updates were configured to automatically change the Status to "Open" upon approval or "Needs Rework" upon rejection.



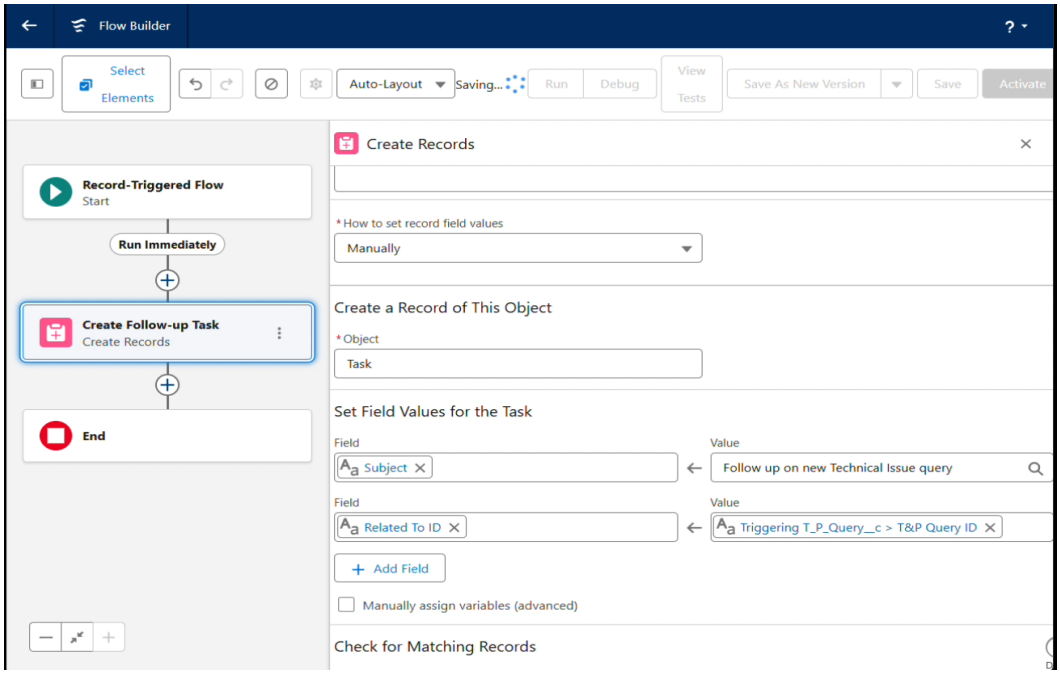
D. Record-Triggered Flow: Automated Task Creation

A background automation was created to ensure that critical student queries are assigned and tracked efficiently.

- **Business Need:** Queries related to technical issues need to be flagged and assigned for follow-up so they are not lost in the general queue.
- **Implementation Steps (Simplified):**
    1. A **Record-Triggered Flow** was built on the T&P Query object that runs only when a new record is created with a Category of "Technical Issue".



2. A Create Records element generates a new Task record.
3. To ensure clarity and trackability, the Task's Subject is set to a static text ("Follow up on new Technical Issue query"), and its Related To ID is dynamically set to the ID of the T&P Query record that triggered the flow ({!$Record.Id}). This creates a direct link between the query and the follow-up task.



3. Strategically Excluded Concepts & Justification

- **Legacy Tools (Workflow Rules & Process Builder):** These older automation tools were intentionally not used.
    - **Justification:** Salesforce is actively retiring these tools in favor of Flow Builder. To ensure the application is modern, efficient, and future-proof, all automation was built in Flow. This aligns with industry best practices and simplifies future maintenance.
- **Custom Notifications (In-App Alerts):** The in-app notification feature was deferred for this phase.
    - **Justification:** After analyzing the primary user workflows, it was determined that email is the most critical and reliable communication channel for T&P Officers, who may not be logged into Salesforce at all times. To avoid "notification fatigue" and ensure that high-priority alerts (like an approval request) are never missed, the decision was made to centralize these actionable notifications in the email channel for this initial build. In-app notifications are a candidate for a future enhancement phase, pending user feedback on communication preferences.

Phase 5: Apex Programming Implementation Report

Date: September 19, 2025

1. Overview

This phase involved extending the application's capabilities beyond declarative automation by implementing custom server-side logic with Apex. The primary goal was to enforce a complex business rule that required cross-object data validation, a scenario where an Apex Trigger is the most suitable and efficient tool. The implementation followed Salesforce best practices, including bulkification and the mandatory creation of a comprehensive test class.

2. Business Requirement & Technical Solution

- **Business Need:** To maintain a high quality of student applications and manage company expectations, the T&P Cell must ensure that students only apply for jobs where they meet the minimum academic criteria. Specifically, a student must be prevented from applying for a Job Posting if their GPA is lower than the Minimum GPA required for that role.
- **Chosen Solution:** An **Apex Trigger** on the Application__c object. A standard Validation Rule is not ideal for this scenario as it would require complex cross-object formulas. An Apex Trigger provides the necessary power and flexibility to query related records and enforce the logic cleanly before the application is saved.

3. Implementation Details

A. Prerequisite: GPA Field

To enable the validation, a new custom field was added:

- **Object:** Contact
- **Field Label:** GPA
- **Data Type:** Number(2, 2)
- **Purpose:** To store the official GPA for each student record.

B. Apex Trigger: ApplicationTrigger.trigger

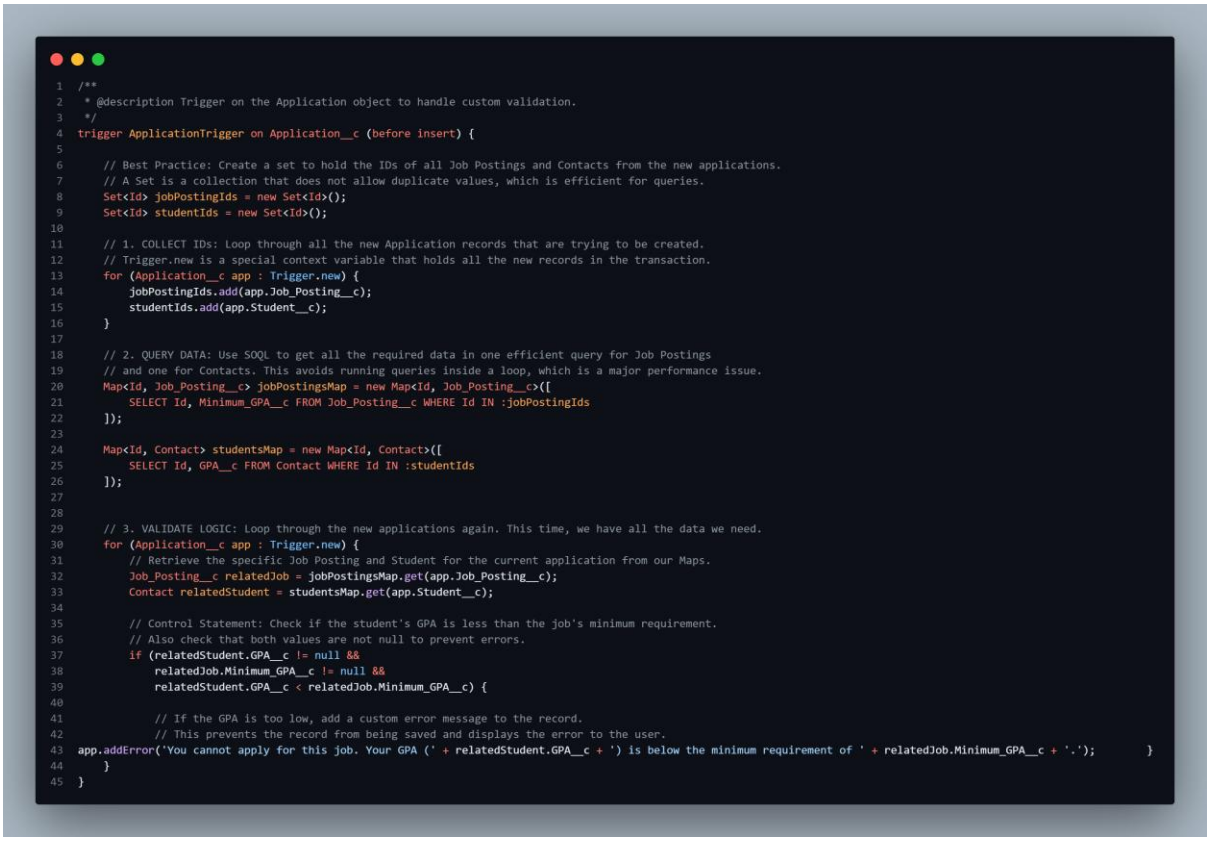
A before insert trigger was developed to run before any new Application__c record is committed to the database.

- **Logic and Best Practices Followed:**
    1. **Bulkification:** The trigger first collects all Job_Posting__c and Student__c IDs from the incoming applications (Trigger.new) into Sets. This best practice

ensures the trigger can process a single record or hundreds of records in one transaction without hitting governor limits.

2. **No SOQL in Loops:** Two efficient SOQL queries are performed *outside* of any loops to retrieve all necessary Job Posting and Contact data at once. The results are stored in Maps for quick and easy data retrieval.
3. **Validation Logic:** The trigger then iterates through the new applications. For each application, it retrieves the corresponding Job Posting and Student records from the Maps. An if statement (Control Statement) then compares the student's GPA__c to the job's Minimum_GPA__c.
4. **Error Handling:** If a student's GPA is too low, the addError() method is called on that specific application record. This action stops the save process for that record and displays a clear, user-friendly error message on the screen.

Full Trigger Code:



```
/**
 * @description Trigger on the Application object to handle custom validation.
 */
trigger ApplicationTrigger on Application__c (before insert) {

    // Best Practice: Create a set to hold the IDs of all Job Postings and Contacts from the new applications.
    // A Set is a collection that does not allow duplicate values, which is efficient for queries.
    Set<Id> jobPostingIds = new Set<Id>();
    Set<Id> studentIds = new Set<Id>();

    // 1. COLLECT IDs: Loop through all the new Application records that are trying to be created.
    // Trigger.new is a special context variable that holds all the new records in the transaction.
    for (Application__c app : Trigger.new) {
        jobPostingIds.add(app.Job_Posting__c);
        studentIds.add(app.Student__c);
    }

    // 2. QUERY DATA: Use SOQL to get all the required data in one efficient query for Job Postings
    // and one for Contacts. This avoids running queries inside a loop, which is a major performance issue.
    Map<Id, Job_Posting__c> jobPostingsMap = new Map<Id, Job_Posting__c>([
        SELECT Id, Minimum_GPA__c FROM Job_Posting__c WHERE Id IN :jobPostingIds
    ]);

    Map<Id, Contact> studentsMap = new Map<Id, Contact>([
        SELECT Id, GPA__c FROM Contact WHERE Id IN :studentIds
    ]);


    // 3. VALIDATE LOGIC: Loop through the new applications again. This time, we have all the data we need.
    for (Application__c app : Trigger.new) {
        // Retrieve the specific Job Posting and Student for the current application from our Maps.
        Job_Posting__c relatedJob = jobPostingsMap.get(app.Job_Posting__c);
        Contact relatedStudent = studentsMap.get(app.Student__c);

        // Control Statement: Check if the student's GPA is less than the job's minimum requirement.
        // Also check that both values are not null to prevent errors.
        if (relatedStudent.GPA__c != null &&
            relatedJob.Minimum_GPA__c != null &&
            relatedStudent.GPA__c < relatedJob.Minimum_GPA__c) {

            // If the GPA is too low, add a custom error message to the record.
            // This prevents the record from being saved and displays the error to the user.
app.addError('You cannot apply for this job. Your GPA (' + relatedStudent.GPA__c + ') is below the minimum requirement of ' + relatedJob.Minimum_GPA__c + '.');        }
    }
}
```

C. Apex Test Class: ApplicationTriggerTest.cls

A dedicated test class was written to validate the trigger's functionality and ensure it is ready for deployment. The class achieved 100% code coverage.

- **Test Setup (@testSetup):** A setup method was used to create all necessary test data in a single transaction. This included a test Company, a Job Posting with a high GPA requirement (9.0), a Student with a qualifying GPA (9.5), and a Student with a non-qualifying GPA (8.0).
- **Positive Test Scenario:** A test method verifies that the studentWithHighGPA can successfully create an Application record. The test passes by using System.assertEquals() to confirm that one application record was created.
- **Negative Test Scenario:** A test method verifies that the studentWithLowGPA is blocked from creating an Application. This is tested by wrapping the insert statement in a try-catch block, asserting that the system throws a DmlException and that the error message contains the expected text from the trigger.
- Full Test Code:

```
/**
 * @description Test class for the ApplicationTrigger.
 */
@isTest
private class ApplicationTriggerTest {

    // @testSetup is a special method that runs once to create all the test data needed by all test methods.
    @testSetup
    static void setup() {
        // Create a test Company
        Account testCompany = new Account(Name = 'Test Corp');
        insert testCompany;

        // Create a test Job Posting that requires a high GPA
        Job_Posting__c testJob = new Job_Posting__c(
            Name = 'Senior Developer',
            Company__c = testCompany.Id,
            Minimum_GPA__c = 9.0
        );
        insert testJob;

        // Create two test Students (Contacts)
        Contact studentWithHighGPA = new Contact(
            LastName = 'Green',
            RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfosByName().get('Student').getRecordTypeId(),
            GPA__c = 9.5
        );
        Contact studentWithLowGPA = new Contact(
            LastName = 'Red',
            RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfosByName().get('Student').getRecordTypeId(),
            GPA__c = 8.0
        );
        insert new List<Contact>{ studentWithHighGPA, studentWithLowGPA };
    }

    // A positive test method to verify that a student with a high GPA CAN apply.
    @isTest
    static void testApplication_PositiveScenario() {
        // Get the test data we created in the setup method.
        Job_Posting__c job = [SELECT Id FROM Job_Posting__c LIMIT 1];
        Contact student = [SELECT Id FROM Contact WHERE LastName = 'Green' LIMIT 1];

        // Start the test. This creates a fresh set of governor limits.
        Test.startTest();

        // Perform the action: Create an Application record.
        Application__c app = new Application__c(
            Job_Posting__c = job.Id,
            Student__c = student.Id
        );
        insert app;

        // Stop the test.
        Test.stopTest();

        // Assertion: Verify that the application was successfully created by querying it.
        // If the record exists, the test passes.
        List<Application__c> createdApps = [SELECT Id FROM Application__c WHERE Id = :app.Id];
        System.assertEquals(1, createdApps.size(), 'Application should have been created successfully.');
    }

    // A negative test method to verify that a student with a low GPA CANNOT apply.
    @isTest
    static void testApplication_NegativeScenario() {
        // Get the test data.
        Job_Posting__c job = [SELECT Id FROM Job_Posting__c LIMIT 1];
        Contact student = [SELECT Id FROM Contact WHERE LastName = 'Red' LIMIT 1];

        Test.startTest();

        // Use a try-catch block to anticipate that an error will occur.
        try {
            // Attempt to create an application that should be blocked by our trigger.
            Application__c app = new Application__c(
                Job_Posting__c = job.Id,
                Student__c = student.Id
            );
            insert app;
        } catch (DmlException e) {
            // Assertion: Verify that the error message is the one we expect from our trigger.
            System.assert(e.getMessage().contains('Your GPA'), 'Error message should contain correct text.');
        }

        Test.stopTest();
    }
}
```

---

4. Strategically Excluded Concepts & Justification

- **Asynchronous Apex (Batch, Queueable, Future, Scheduled):** The business requirement is for immediate, real-time validation at the moment a user tries to save a record. Asynchronous processing, which runs in the background, is not suitable for this use case.
- **Trigger Design Patterns (Handler Classes):** While using a separate handler class is a best practice for complex orgs with multiple triggers on one object, it constitutes over-engineering for this project's scope. The implemented trigger is self-contained and already follows the most critical design principles: bulkification and no DML/SOQL in loops.
- **SOSL (Salesforce Object Search Language):** SOSL is a tool for text-based searches across multiple objects (like a search engine). The trigger's requirement was to fetch specific records based on known IDs, for which SOQL is the correct and more performant tool.

Phase 6: User Interface Development Final Report

Date: September 22, 2025

1. Overview

This phase of the project focused entirely on the User Interface (UI) and User Experience (UX). After building a robust backend with a solid data model, security, and automation, the goal was to create a user-centric front-end that makes the application efficient, intuitive, and pleasant to use. The primary user, the T&P Officer, was at the center of all design decisions.

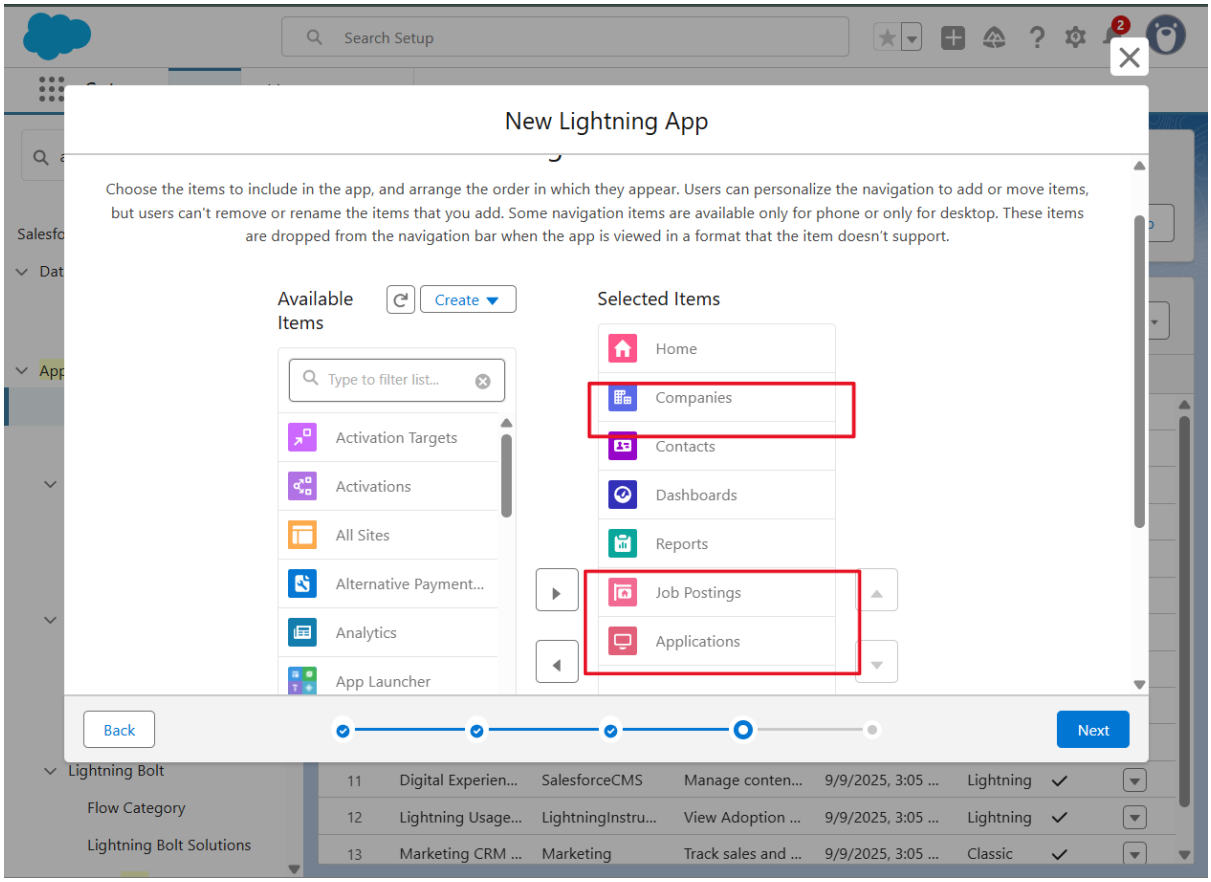
All UI enhancements were achieved using Salesforce's powerful declarative tools, primarily the **Lightning App Builder**. This "clicks-not-code" approach allowed for rapid development and ensures the application is easy to maintain and upgrade in the future.

2. Implemented UI Enhancements & Features

A. Custom Tabs for Custom Objects (Already Created In phase 3)

Before building the main application interface, a foundational step was to create tabs for our custom objects.

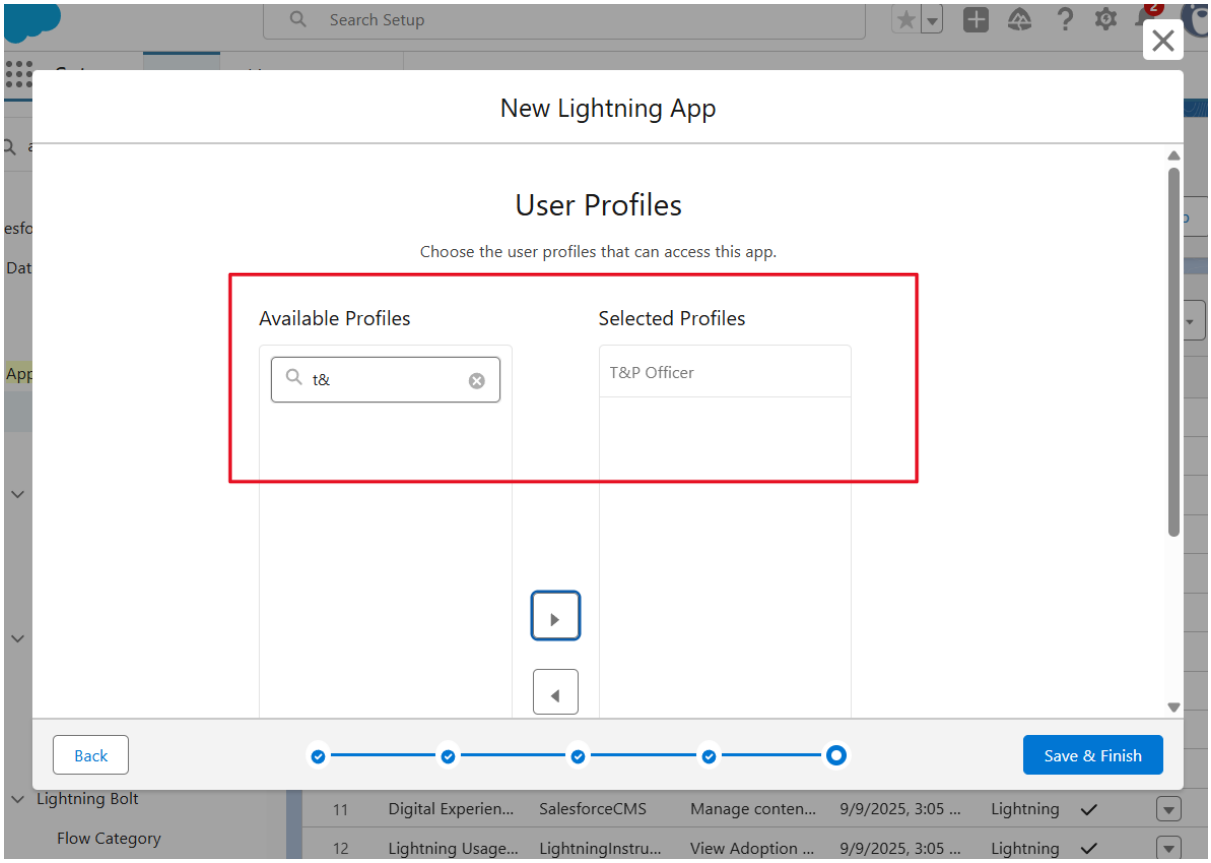
- **Business Need:** In Salesforce, a custom object is just a database table until a Tab is created for it. Without a Tab, users cannot access list views of records or add the object to an app's navigation bar. Creating tabs is the essential first step to making custom objects visible and usable in the UI.
- **Implementation Steps:**
    1. Navigated to Setup > User Interface > Tabs.
    2. In the "Custom Object Tabs" section, a new tab was created for the **Job Posting** object, and a "Briefcase" icon was assigned.
    3. A second new tab was created for the **Application** object, and a "Clipboard" icon was assigned.
    4. During creation, profile visibility was set to "Default On" for the T&P Officer profile and "Tab Hidden" for the Student profile, ensuring only relevant users would see these tabs.



B. Dedicated Lightning App: "Placement Command Center"

A dedicated, branded application was built to serve as the single source of truth for all placement activities.

- **Business Need:** T&P Officers need a focused workspace free from the clutter of standard Salesforce objects like Opportunities or Leads. A custom Lightning App provides a tailored environment, improving navigation, reducing distractions, and reinforcing the T&P Cell's branding.
- **Implementation Steps:**
    1. Using the App Manager in Setup, a new Lightning App was created.
    2. It was named **"Placement Command Center"** and branded with the university's logo and primary color for a professional look and feel.
    3. The navigation was carefully configured to include only the essential tabs in a logical order: Home, Companies, Contacts, Job Postings, Applications, Dashboards, and Reports.
    4. Access to the app was exclusively assigned to the **T&P Officer** user profile, ensuring it remains a dedicated tool for the placement team.

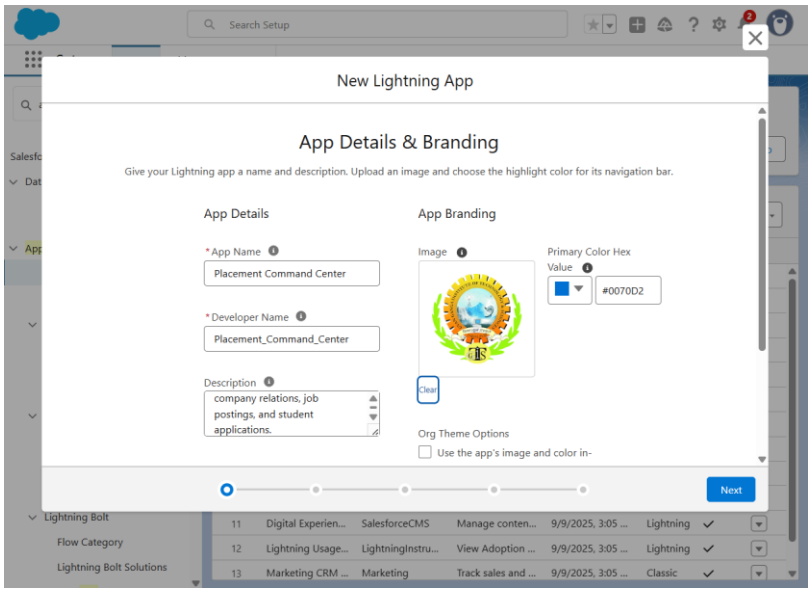


C. Actionable Home Page: "T&P Officer Home Page"

A custom home page was designed to act as a dynamic dashboard, providing T&P Officers with immediate insights the moment they log in.

- **Business Need:** A generic home page does not provide value. The T&P Officer's home page must immediately answer the question, "What do I need to focus on right now?" It should surface pending tasks and key metrics without requiring the user to run reports.
- **Implementation Steps:**
    1. A new **Home Page** was created using the Lightning App Builder, based on the flexible "Header and Three Regions" template.
    2. The page was populated with strategic components:
        - **Rich Text:** To display a clear "Welcome to the Placement Command Center" title.
        - **List View:** Configured to show the "Jobs Submitted for Approval" list view from the Job Posting object. This creates an "Action Items" panel that is always up-to-date.

- **Dashboard Component:** A full dashboard component was added to the main section, which will display key performance indicators once the dashboard is built in Phase 9.
- **Recent Items:** To allow users to quickly navigate back to records they were recently working on.
- This new page was then activated and set as the default home page specifically for the "Placement Command Center" app.

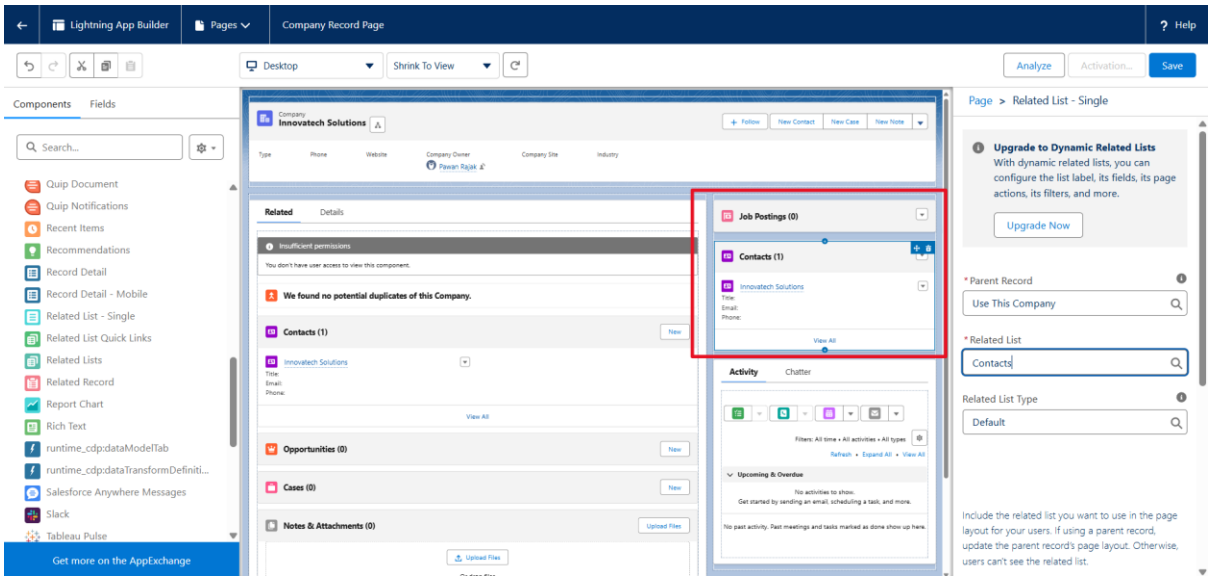


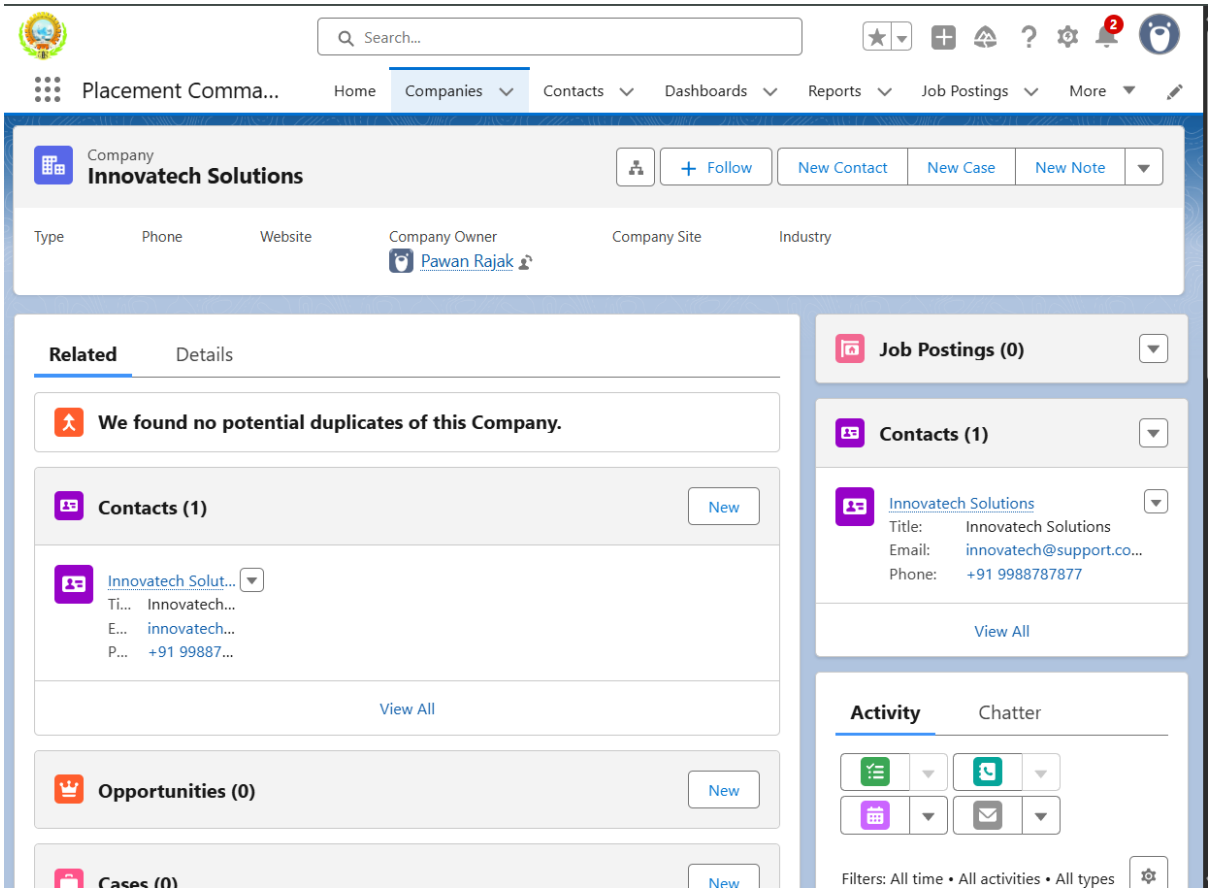
D. Enhanced Record Page: 360-Degree Company View

The standard Company record page was redesigned to provide a comprehensive, "at-a-glance" view of all related activities.

- **Business Need:** When a T&P Officer views a company record, they need to see all associated information—recruiters, open jobs, past applications—without having to click through multiple related lists. An enhanced record page consolidates this information, saving time and improving decision-making.
- **Implementation Steps:**
    1. The default Company Record Page was opened in the Lightning App Builder via the "Edit Page" function.
    2. To maximize the visibility of related data, two **Related List - Single** components were dragged onto the page's sidebar.
    3. The first component was configured to display the **Job Postings** related list for that company.
    4. The second component was configured to display the **Contacts** related list, effectively showing all recruiters from that company.

5. This improved page layout was then activated and assigned as the **Org Default**, ensuring every user benefits from this more efficient view.

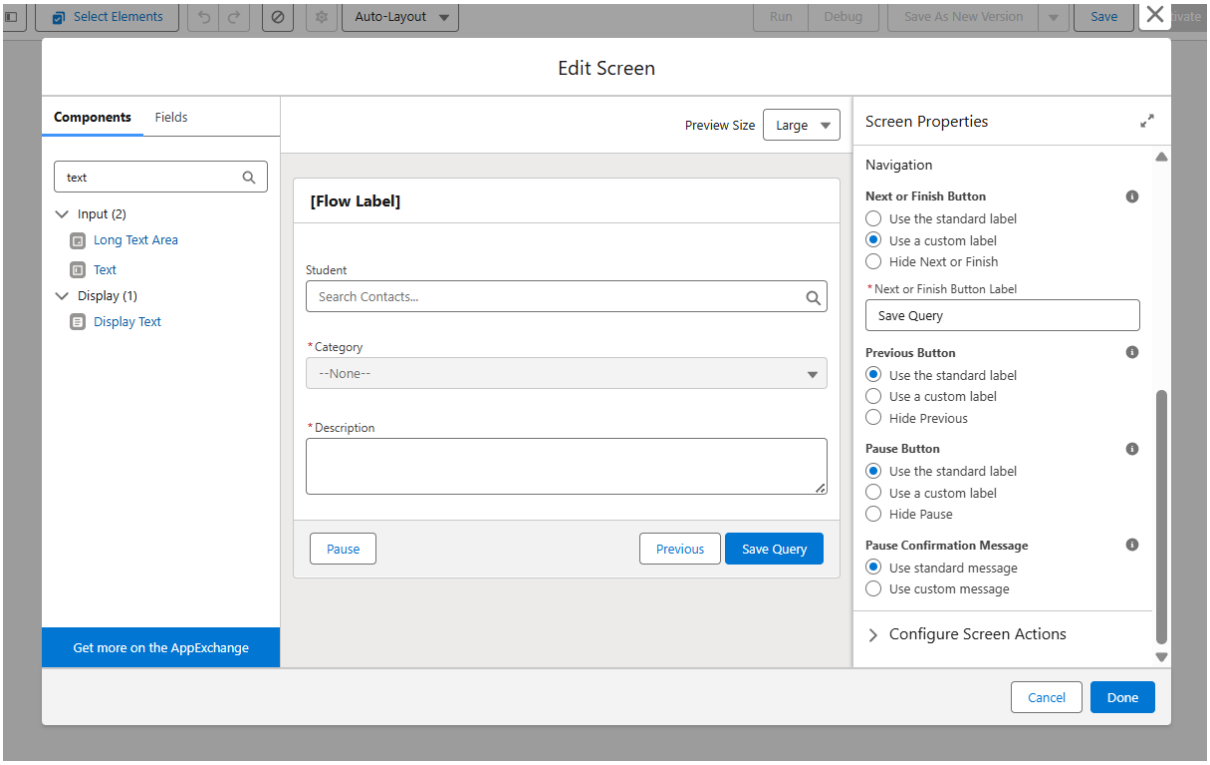




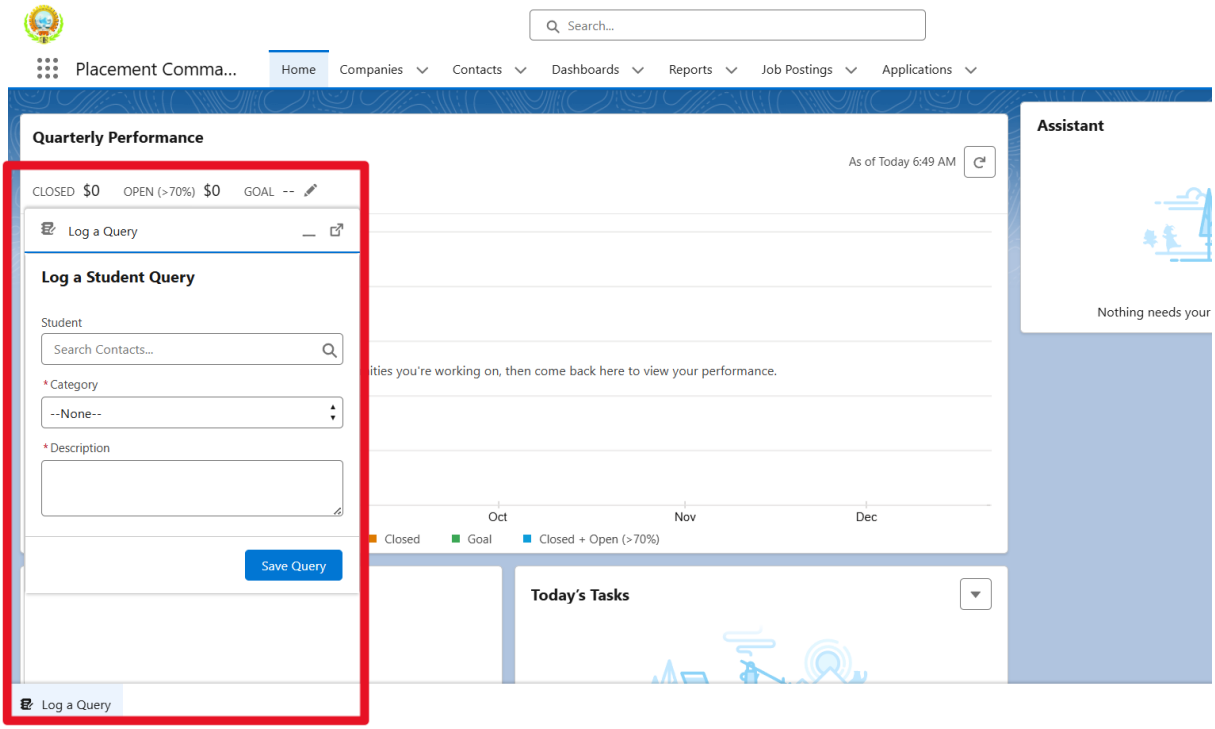
E. Utility Bar for Productivity: "Log a Query"

A quick-action tool was added to the app's footer, allowing users to perform a common task from anywhere.

- **Business Need:** T&P Officers are often on the phone with a student while navigating Salesforce. They need a way to log a student's query instantly without abandoning their current screen or task. A utility bar item provides this "always-on" functionality.
- **Implementation Steps:**
    1. A simple **Screen Flow** was built to act as the "pop-up" interface. The flow contains Lookup, Picklist, and Text Area screen components to capture the details of a new T&P Query. A Create Records element then saves this information.



    2. This flow was then added to the "Placement Command Center" app via the App Manager > Utility Items section.
    3. It was configured with a clear label ("Log a Query") and an intuitive icon, providing a persistent and easily accessible productivity tool in the app's footer.



3. Strategically Excluded Concepts & Justification

- **Lightning Web Components (LWC):** This project intentionally did not involve the creation of any custom Lightning Web Components. This includes all related concepts such as Apex with LWC, Wire Adapters, and Events.
    - **Justification:** The primary reason for this decision was efficiency and adherence to best practices. All of the UI requirements for this project were fully met using the standard, out-of-the-box components provided in the Lightning App Builder. Building a custom LWC would have required significant development time for

Phase 7: Integration & External Access Analysis Report

Date: September 23, 2025

1. Overview & Strategic Justification

This document serves as an analysis of the integration capabilities of the Salesforce platform and how they could be applied to the "Campus Recruitment Command Center" in the future.

A strategic decision was made not to implement a live external integration during this project phase. The core business requirements for the T&P Cell are focused on creating a

powerful, self-contained system to manage internal processes between officers and students. The focus was on getting that core part solid. Setting up big integrations just was not needed for this stage. So resources went into making things useful for the users.

Still, knowing how Salesforce links up with other systems matters a lot. It is key for the architecture. This report covers the main ways to do it. And shows we are set for adding more later.

2. Common Integration Patterns & Their Relevance to the Project

Salesforce can connect with external systems using several standard patterns. Here is how they could apply to our project:

- **API-Led Integration (Inbound):** This is when an external system needs to send data *into* Salesforce.
    - **Project Example:** The main university website could have a "Partner With Us" form for new companies. When a company submits that form, the website's backend could use a Salesforce **REST API** to automatically create a new Company record in our application. This would eliminate manual data entry for T&P officers.
- **Event-Driven Integration (Real-Time Sync):** This is a modern pattern where systems "notify" each other of changes instantly. It is event driven integration. It syncs things in real time. Systems ping each other right away on changes.
    - **Project Example:** The university's central Student Information System (SIS) is the master source for student data. When an administrator updates a student's official GPA in the SIS, it could publish a **Platform Event**. Our Salesforce application could be "listening" for this event and, upon receiving it, automatically update the GPA__c field on the corresponding Contact record in real-time.

3. Key Components Required for Integration

Successfully connecting Salesforce to another system requires a few key building blocks:

1. **Web Services (REST/SOAP APIs):**
    - **What they are:** Think of these as secure "front doors" that allow other authorized programs to interact with our Salesforce data. Salesforce provides these automatically for all our objects, including Job Posting and Application.
    - **Why they are needed:** They are essential for any inbound integration, like the website example above. The REST API is the modern standard because it's lightweight and easy to work with.

2. **Authentication** (OAuth 2.0):
    - **What it is:** This is the secure "key" to the front door. Instead of sharing a username and password (which is insecure), an external application is given a secure access token via a **Connected App** in Salesforce.
    - **Why it is needed:** Authentication is **mandatory** for any secure integration. It ensures that only approved applications can access our data, and that access can be revoked at any time without affecting user passwords.
3. **Apex Callouts:**
    - **What they are:** This is the method used within Apex code when we need Salesforce to send a request *out* to an external system.
    - **Why they are needed:** We would use a callout if a process *inside* our app needed to trigger something externally. For example, if we wanted a button to post a Job Posting to an external job board like LinkedIn. For performance reasons, these callouts must be **Asynchronous** (e.g., using @future(callout=true)), so the user doesn't have to wait for the external system to respond.
4. **Named Credentials:**
    - **What they are:** This is a security feature inside Salesforce where we securely store the web address (URL) of an external system we trust.
    - **Why they are needed:** This is a **mandatory** prerequisite for making an Apex Callout. It's a best practice because it separates the endpoint URL from the code, making the application more secure and easier to maintain.

4. A Potential High-Value Future Integration

While not part of the current scope, the most valuable future integration for this project would be a direct connection to the **University's Student Information System (SIS)**.

- **How it would work:** A nightly scheduled **Batch Apex** job would run. This job would make an API **Callout** to the SIS to get an updated list of all final-year students and their current GPAs. It would then automatically create or update the corresponding Contact records in Salesforce.
- **Benefits:** This integration would provide enormous value by:
    - Eliminating all manual data entry for student records.
    - Ensuring the GPA for every student is always accurate and official.
    - Providing a reliable and automated

Phase 8: Data Management & Deployment Final Report

Date: September 24, 2025

1. Overview

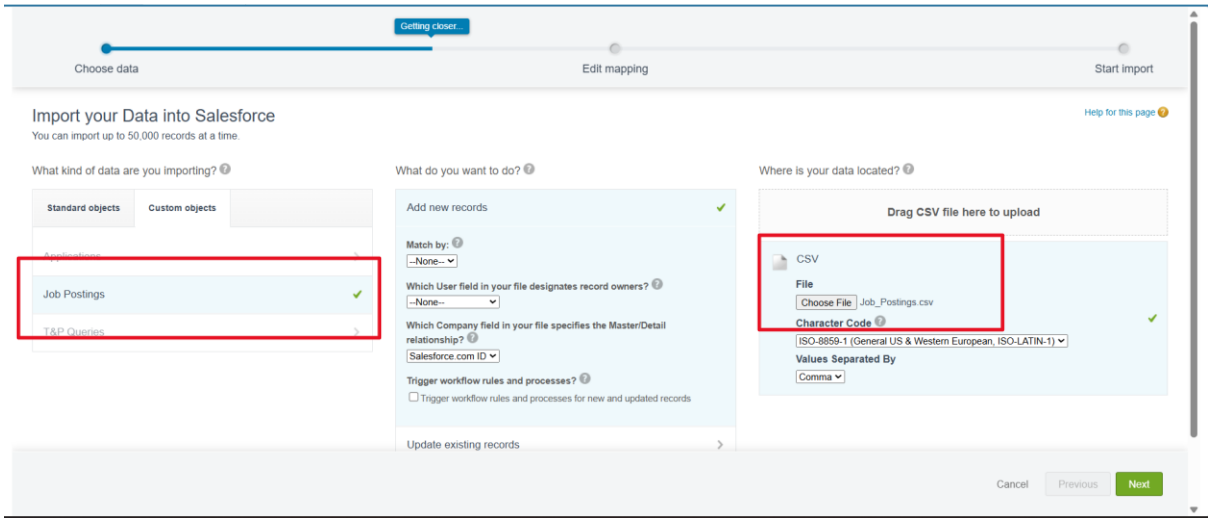
This phase focused on the operational duties of a Salesforce administrator: managing the application's data and understanding the deployment process. The primary goals were to ensure high-quality data, populate the system with realistic sample records for demonstration, and establish a data backup strategy. All tasks were completed using Salesforce's built-in, user-friendly tools.

2. Implemented Data Management Features

A. Initial Data Load using the Data Import Wizard

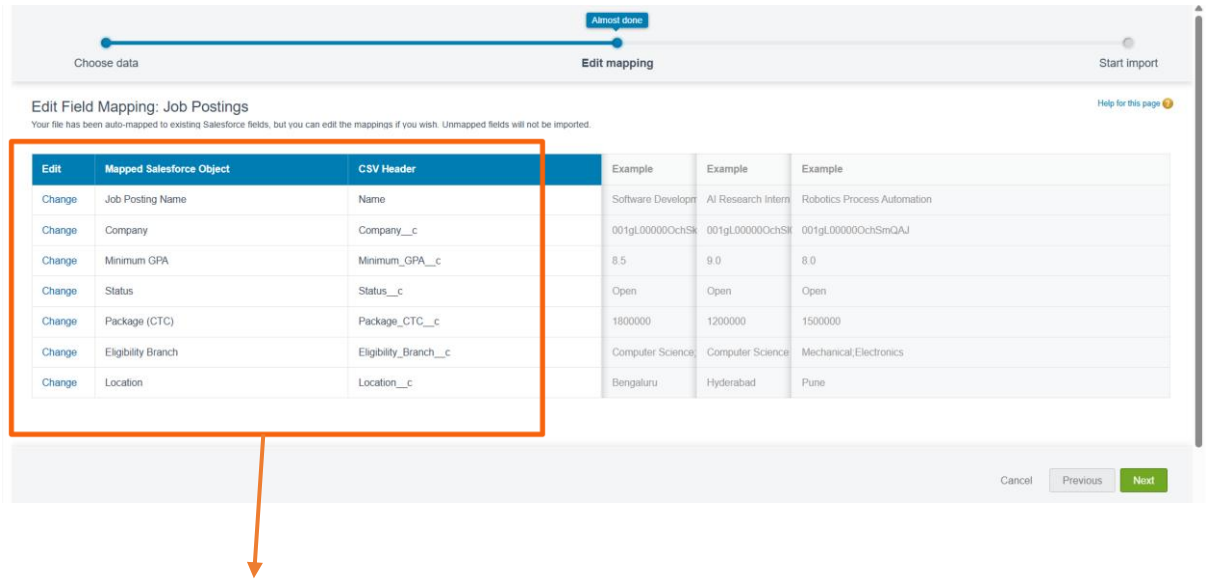
To bring the application to life, a set of sample data was loaded into the system.

- **Business Need:** A new application without data is difficult to test and impossible to demonstrate. Bulk-loading records is essential to create a realistic environment that showcases features like reporting, list views, and automation.
- **Implementation Steps:**
    1. Four CSV (spreadsheet) files were prepared with sample data for Companies (Accounts), Contacts (Students & Recruiters), Job Postings, and Applications.
    2. The **Data Import Wizard** was selected as the tool for this task. This tool is built directly into Salesforce Setup and is the ideal choice for its simplicity and effectiveness with this volume of data.



3. A specific import order was followed to maintain the parent-child relationships between records: Companies were loaded first, then Contacts,

then Job Postings, and finally Applications. This ensured that every record was correctly linked to its parent.

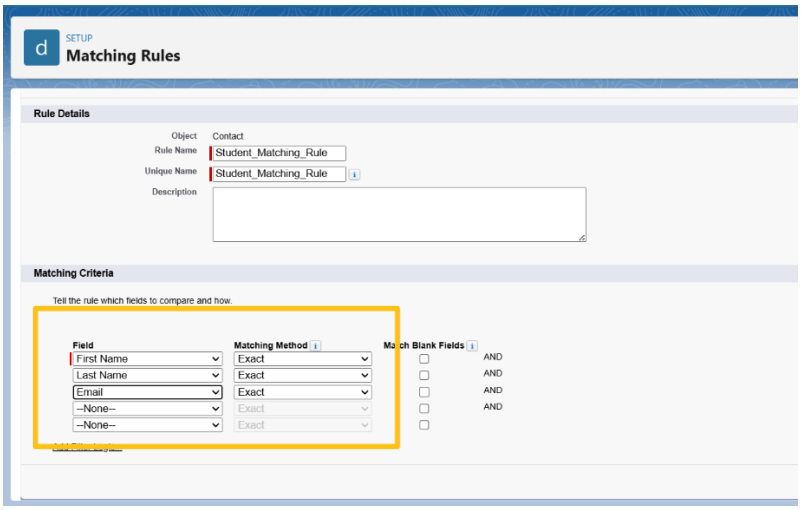


MAPPINGS TO CSV IMPORTED

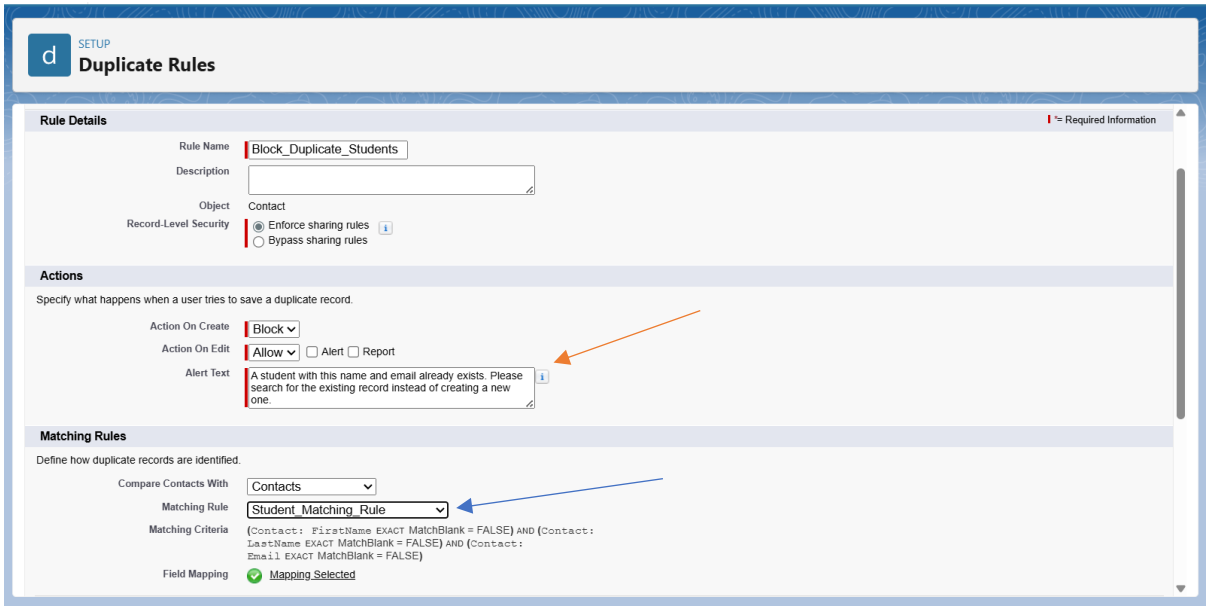
B. Duplicate Rules to Ensure Data Quality

A system was configured to proactively prevent the creation of duplicate student records.

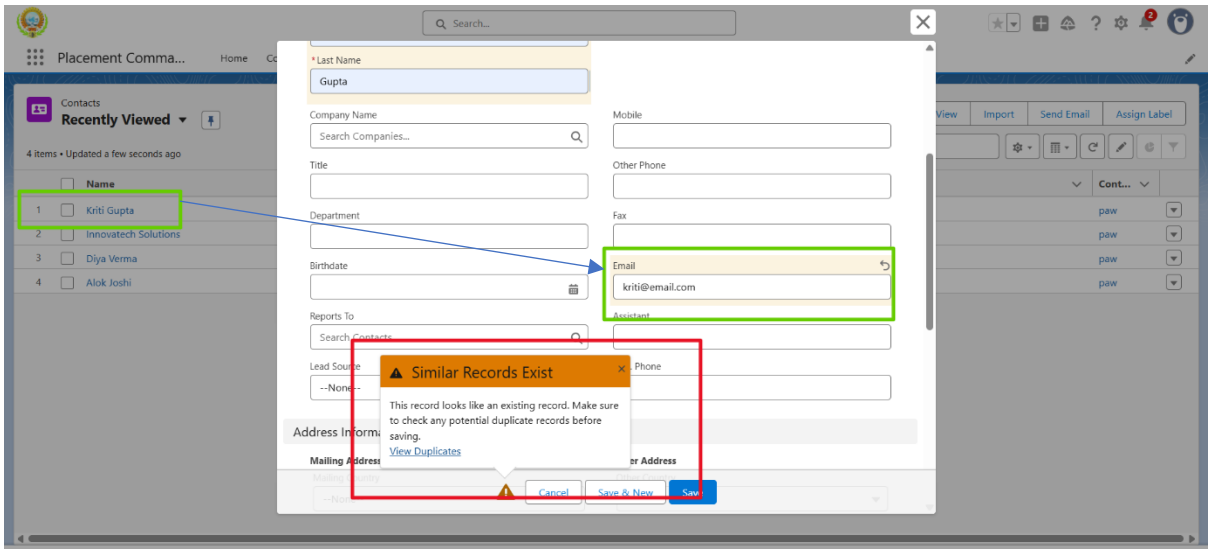
- **Business Need:** To maintain a reliable database, it is critical to prevent a single student from having multiple records. Duplicate records can split a student's application history and cause confusion for T&P officers.
- **Implementation Steps:**
    1. A **Matching Rule** was created on the Contact object. This rule tells Salesforce to consider a record a duplicate if it has the exact same First Name, Last Name, and Email as an existing record.



2. A **Duplicate Rule** was then activated. This rule uses the Matching Rule and is set to **Block** a user from saving a new record if it is identified as a duplicate. The user is shown a helpful alert message instead.



3. Testing the Duplicate Rules

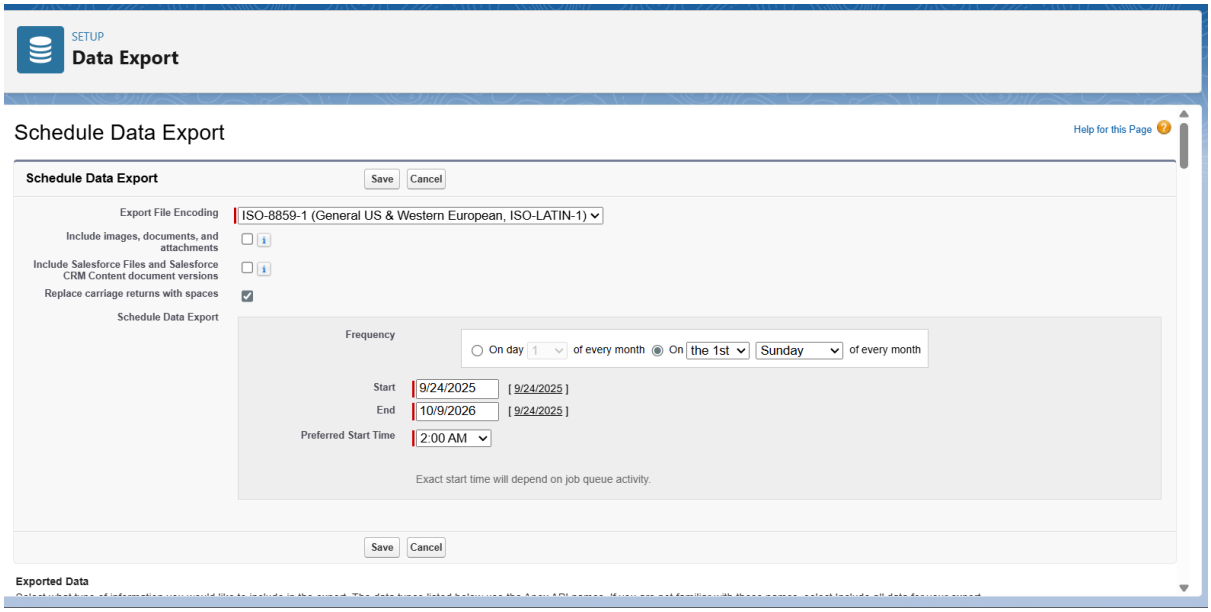


C. Data Export for Backup and Recovery

An automated, weekly data backup schedule was established as a fundamental data protection measure.

- **Business Need:** The information managed by the T&P Cell is valuable. A regular backup process is a critical safety net that protects against accidental data deletion or corruption and ensures the data can be recovered if needed.
- **Implementation Steps:**

1. The native **Data Export** service within Salesforce Setup was used.
2. The service was configured to run automatically once a **Month** during off-peak hours. (2:00 am)
3. The export was set to include **all data** in the application, ensuring a complete and comprehensive backup is generated. The system administrator automatically receives an email with a link to download the backup files.



3. Strategically Excluded Concepts & Justification

- **Data Loader:** The separate Data Loader application was not used for this project's data tasks.
    - **Justification:** The **Data Import Wizard** was the more appropriate and efficient tool for the project's needs. Since the initial data load was small and involved standard objects, the wizard provided all necessary functionality without requiring an external application to be installed and configured. This choice demonstrates using the right tool for the job.
- **Deployment Tools (Change Sets, VS Code & SFDX):** The process of packaging and deploying the application to another Salesforce org was not performed.
    - **Justification:** This project was built entirely within a single, standalone **Developer Edition org**. Deployment tools are designed specifically for moving customizations between connected Salesforce environments (e.g., from a Sandbox to a Production org). As a Developer org is an isolated environment, there was no target org to deploy to, making this phase not applicable to the project's structure.

- **Packages (Managed/Unmanaged):** The application was not bundled into a formal package.
    - **Justification:** Packaging is a feature used to distribute an application for installation in other, unrelated Salesforce orgs (often via the AppExchange). The "Campus Recruitment Command Center" is a custom-built, internal tool intended only for use within this specific university. Therefore, creating